

并发理论课程大作业

跨组织协作业务流程系统的 行为分析与优化

实验指导书

复旦大学计算与智能创新学院
并发理论课程课程大作业组
指导教师：Leon

版本：2026 年 4 月 4 日

目录

I	理论基础与环境搭建	1
1	绪论	3
1.1	课程大作业概述	3
1.2	实验小组划分	4
1.3	理论框架和系统架构	4
1.4	整体验证与优化架构	5
2	环境搭建与工具链安装	7
2.1	实验目的	7
2.2	实验材料	8
2.2.1	硬件环境要求	8
2.2.2	软件材料清单	9
2.3	实验步骤	9
2.3.1	Java 开发环境配置	9
2.3.2	Node.js 环境配置	10
2.3.3	Camunda 8 Run 的安装与配置	10
2.3.4	Camunda Desktop Modeler 安装与配置	12
2.3.5	Camunda 8 可用性测试	14
2.3.6	CPN Tools 安装	17
2.3.7	CPN IDE 安装	20
2.3.8	mCRL2 安装	21
2.3.9	KeYmaera X 安装与 Mathematica 配置	23
2.3.10	辅助工具安装	26

2.4	预期结果	27
2.5	检验方法	27
2.5.1	环境整体检验脚本	27
2.5.2	集成测试：完整的简单协作流程	30
2.6	常见问题与解决方法	31
II	业务流程建模	33
3	集装箱出口协作流程	35
3.1	流程概述	35
3.2	完整消息流规范	35
3.3	核心并行汇聚点	36
3.3.1	货代汇聚点：信息完整性的保障	37
3.3.2	码头汇聚点：资源调度的前提	39
3.3.3	海关汇聚点：监管决策的基础	40
3.3.4	汇聚点间的关联与影响	41
3.3.5	对流程设计的启示	41
III	并发性验证 (CPN Tools)	43
4	着色 Petri 网建模	45
4.1	理论基础	45
4.1.1	CPN 基本元素	45
4.2	颜色集定义	45
IV	消息一致性验证 (mCRL2)	47
5	进程代数建模	49
5.1	理论基础	49
5.1.1	mCRL2 核心概念	49

5.2	数据类型定义	49
5.3	挑战场景：网关隐藏	50
5.3.1	场景 A：XOR 网关隐藏（消息时有时无）	50
5.3.2	场景 B：并行网关隐藏（消息顺序不确定）	51
V	信信息物理融合系统验证（KeYmaera X）	53
6	码头 CPS 建模与微分动态逻辑	55
6.1	理论基础	55
6.1.1	dL 核心概念	55
6.2	码头 CPS 场景描述	55
6.3	经济优化场景	56
6.4	带经济约束的 AGV 运动模型	56
VI	架构对比	59
7	固定协作架构（编排器/适配器）	61
7.1	架构概述	61
8	即时协作架构（编舞）	63
8.1	架构概述	63
8.2	基于 KPI 的动态参与方选择	63
9	对比分析与结果	65
9.1	多维对比	65
9.2	基于 KPI 的动态选择收益	66
9.3	适用场景建议	67
	索引	69

插图

1.1	集装箱出口国内段端到端协作业务流程	3
1.2	集成三种形式化方法的整体验证与优化架构	5
2.1	Camunda 登录界面	11
2.2	Camunda Operate 监控界面	12
2.3	设置 Camunda Modeler 以与 Camunda 建立连接	13
2.4	测试用业务流程模型	14
2.5	vsCode 终端启动 Worker 运行结果	15
2.6	Camunda Operate 界面查看流程实例状态	15
2.7	协作流程示意图	16
2.8	vsCode 终端启动 Worker 运行结果	16
2.9	CPN Tools 主界面	18
2.10	CPN Tools 主界面	19
2.11	CPN IDE 主界面	20
2.12	CPN IDE 的 Simulation	21
2.13	mCRL2 命令行运行截图	23
2.14	KeYmaera X 启动界面截图	24
2.15	KeYmaera X 模型编辑	25
2.16	KeYmaera X 证明界面截图	26
3.1	包含所有 9 个参与方和部分消息流的完整 BPMN 协作图	35
3.2	数据对象、汇聚点与业务目标的关联关系	37
3.3	货代汇聚点：舱单与 EIR 的同步	37
3.4	码头汇聚点：三要素同步	39
3.5	海关汇聚点：多源信息融合	40

6.1	自动化集装箱码头布局，显示桥吊、AGV 和龙门吊	56
7.1	带有中央编排器和参与方适配器的固定协作架构	61
8.1	点对点直接消息交换的即时协作架构	63
8.2	基于 KPI 评估在多个船代实例之间进行动态选择	64
9.1	基于关系稳定性和可靠性要求的架构选择矩阵	67

表格

1.1	核心理论框架	4
2.1	样例硬件环境配置	8
2.2	软件材料清单	9
2.3	Camunda Modeler 连接配置参数	13
2.4	常见问题及解决方法	15
2.5	常见安装问题及解决方法	31
3.1	完整消息流规范	36
9.1	基于形式化验证结果的全面架构对比	66
9.2	100 个订单中基于 KPI 动态选择的改进	67

第 I 部分

理论基础与环境搭建

第 1 章 绪论

当代商业环境已从单一企业的独立运营转向多企业协同的生态系统。供应链全球化、数字经济兴起和服务外包普及，使得任何企业都无法孤立存在。企业必须与供应商、合作伙伴、服务商和客户建立紧密的协作关系，才能在竞争中生存和发展^[1]。这种转变催生了对跨组织业务流程管理的系统化研究。

跨组织协作涉及多个独立组织的协同工作，各组织拥有自主的业务流程和信息系统，但必须通过标准化的接口和协议实现互联互通^[2]。在不同组织间协调流程执行、交换业务数据、保证一致性和可靠性，成为理论研究和工业实践共同关注的核心问题。

1.1 课程大作业概述

本大作业旨在集装箱出口国内段端到端协作业务流程 (见图 1.1) 的行为分析和优化。具体地，采用三种形式化验证工具与 Camunda 8 集成形式化方法建模、分析、验证和优化跨组织协作中的集装箱出口流程。系统阐述跨组织协作的理论基础和实践框架。

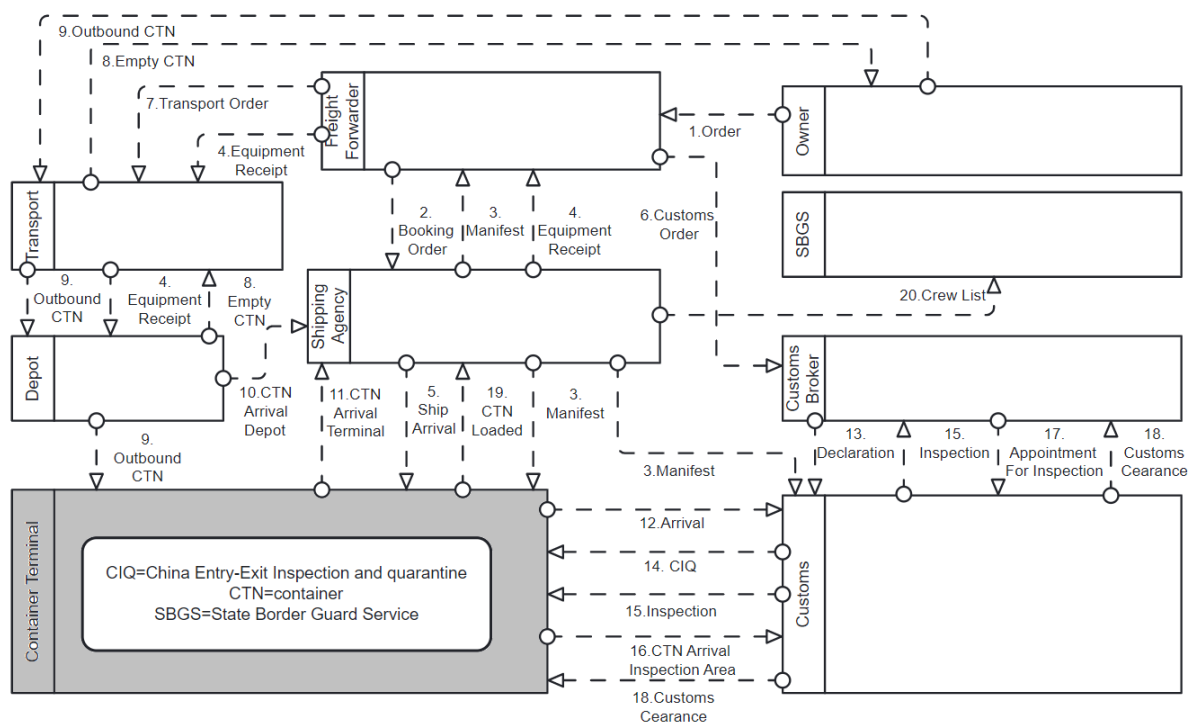


图 1.1: 集装箱出口国内段端到端协作业务流程

集装箱出口涉及货主、货代、船代、报关行、海关、边防、码头、车队、货场等九

个独立组织、二十八条消息流、四十四四个服务任务和三个关键的并行汇聚点，是跨组织协作的典型场景^[3]。

课程大作业结合这个具体案例，采用下述三种形式化方法和工具加以分析

- **CPN Tools**^[4]: 着色 Petri 网，用于并发性验证、死锁检测和整体协作流程的性能分析
- **mCRL2**^[5]: 进程代数，用于消息一致性检查、行为等价性验证和交互协议的验证
- **KeYmaera X**^[6]: 微分动态逻辑，用于信息物理融合系统（Cyber-Physical System, CPS）验证，特别是码头操作的经济约束建模与优化

使学生获得对以下问题的洞察：跨组织协作为什么成为必然？其核心内涵是什么？如何理解，分析和优化跨组织流程协作的行为？当前面临哪些挑战？为达成上述目标，又有哪些技术标准和架构模式可供选择？

1.2 实验小组划分

考虑到选课人数，计划分两个平行队伍，每个队伍实施同样的系统，即这个班级有两队人完成同样的工作。在每个队伍中，再进一步划分为四组：每种形式化方法各一组，Camunda 部署和协作流程建模一组。一共有八个组，每个组自己选择一位组长，相当于项目经理，统筹本组的各项事务，记录小组成员的贡献（最后课程大作业个人的分数比例有组长负责裁定）。

1.3 理论框架和系统架构

本课程大作业的理论基础建立课程的五部核心教材之上，它们为本课程大作业提供了方法论框架。

表 1.1: 核心理论框架

专著	方法	本课程大作业中的应用
Kunze & Weske (2016) ^[7]	行为模型	流程抽象、BPMN 到形式化模型转换、接口行为建模
Jensen & Kristensen (2009) ^[4]	着色 Petri 网	整体协作流程建模、并发性验证、死锁检测
Atif & Groote (2023) ^[5]	mCRL2	消息序列一致性、XOR/并行网关隐藏检测、行为等价性
Weske (2024) ^[1]	BPMN	业务流程建模、Camunda 8 部署、流程编排与编舞对比
Platzer (2018) ^[6]	微分动态逻辑	码头 CPS 建模、资源分配优化、经济协议验证

1.4 整体验证与优化架构

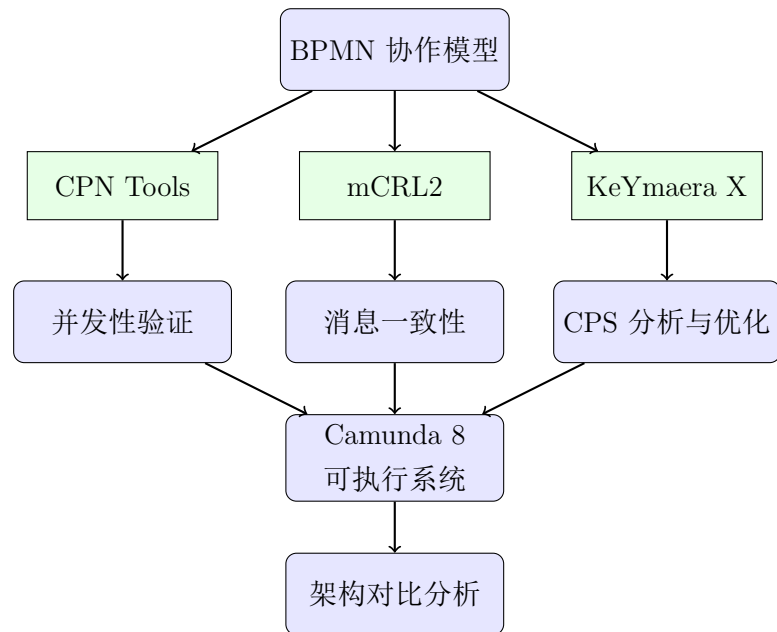


图 1.2: 集成三种形式化方法的整体验证与优化架构

第 2 章 环境搭建与工具链安装

2.1 实验目的

本章旨在为跨组织协作流程的建模、仿真与验证搭建完整的软硬件实验环境。具体包括：

- 配置 Java 开发环境：安装 OpenJDK 21，为 Camunda 8 提供运行基础
- 配置 Node.js 开发环境
- 部署 Camunda 8 Run：包括轻量级本地流程执行环境 Camunda Run 8 和 Camunda Desktop Modeler。配置流程建模工具与运行环境的集成
- 安装形式化验证工具：完成 CPN Tools、mCRL2 和 KeYmaera X 的安装
- 验证工具链功能：通过简单协作流程示例检验各工具安装正确性及协同工作能力

2.2 实验材料

2.2.1 硬件环境要求

表 2.1: 样例硬件环境配置

类别	配置要求
最低配置	
操作系统	Windows 11 / macOS Tahoe 26 / Ubuntu 24.04
处理器	Intel i5 或同等性能 (4 核心)
内存	8 GB
硬盘空间	20 GB 可用空间
推荐配置	
作系统	Windows 11 / macOS Tahoe 26 / Ubuntu 24.04
处理器	Intel i7 或同等性能 (8+ 核心)
内存	16 GB ^a
硬盘空间	50 GB 可用空间 (SSD)

^a 16 GB 内存适用于中等规模的状态空间探索，更大规模验证（如完整集装箱出口流程）可能需要 32 GB 以上。本手册将以 Thinkapad L13 上 Windows 11 操作系统为工作环境为例，介绍安装过程。其他机器或操作系统的配置需做相应的调整

2.2.2 软件材料清单

表 2.2: 软件材料清单

软件名称	版本	下载地址/来源
OpenJDK	21 LTS	https://adoptium.net
Node.js	24.14.0	https://nodejs.org
Camunda 8 Run	8.8.11	https://camunda.com/download/run
Camunda Modeler	5.44.0	https://camunda.com/download/modeler
CPN Tools	4.0.1	http://cpntools.org
mCRL2	2025.07	https://www.mcrl2.org
KeYmaera X	5.2.1	https://keymaerax.org
Mathematica ^a	12.1+	https://www.wolfram.com/mathematica
Docker Desktop	4.63.0+	https://www.docker.com/products/docker-desktop
Git	2.40+	https://git-scm.com
Visual Studio Code	1.110.1	https://code.visualstudio.com
LaTeX 发行版	MiKTeX/TeX Live	https://miktex.org 或 https://tug.org/texlive

^a Mathematica 为可选安装，用于增强 KeYmaera X 的算术求解能力。如无许可证，可使用 KeYmaera X 内置的简化求解器。

2.3 实验步骤

2.3.1 Java 开发环境配置

Camunda 8 Run 基于 Java 运行，因此首先需要安装 Java 开发环境。

1. 下载 OpenJDK 21

访问 Adoptium 官方网站 (<https://adoptium.net>)，选择 OpenJDK 21 (LTS) 版本，根据操作系统下载相应的安装包：

- Windows: `OpenJDK21U-jdk_x64_windows_hotspot_21.0.10.msi`
- macOS: `OpenJDK21U-jdk_aarch64_mac_hotspot_21.0.10.pkg`
- Linux: `OpenJDK21U-jdk_x64_linux_hotspot_21.0.10.tar.gz`

2. 安装 JDK

- **Windows:** 双击运行 MSI 安装包，按提示完成安装。例如，缺省安装路径为 `C:\Program Files\Java\jdk-21.0.10`；
- **macOS:** 双击运行 PKG 安装包，按提示完成安装。JDK 将被安装到 `/Library/Java/JavaVirtualMachines/` 目录；
- **Linux:** 解压 tar.gz 文件到 `/usr/lib/jvm/` 目录。

3. 配置环境变量

- **Windows:**

- (a) 右键点击"此电脑" "属性" "高级系统设置" "环境变量"
- (b) 在"系统变量"中新建变量 `JAVA_HOME`, 取值为 JDK 安装路径(如 `C:\Program Files\Java\jdk-21.0.10`)
- (c) 编辑 `Path` 变量, 添加 `%JAVA_HOME%\bin`
- **macOS/Linux:**
 - (a) 编辑 `/.bashrc` 或 `/.zshrc` 文件, 添加:

```
export JAVA_HOME=/usr/lib/jvm/default-java
export PATH=$JAVA_HOME/bin:$PATH
```
 - (b) 执行 `source ~/.bashrc` 使配置生效

4. 验证安装

打开命令行终端, 执行命令 `java -version` 验证, 如果正确显示 JDK 21 版本信息, 无错误提示, 便是成功。

2.3.2 Node.js 环境配置

在 Windows 11 上配置 Node.js 开发环境:

1. 下载 Node.js LTS 版本安装程序 (<https://nodejs.org/>)
2. 运行安装程序, 确保勾选“npm package manager”和“Add to PATH”
3. 打开命令提示符, 验证安装:

```
node --version
npm --version
```

4. (可选) 配置 npm 镜像加速:

```
npm config set registry https://registry.npmmirror.com
```

2.3.3 Camunda 8 Run 的安装与配置

Camunda Run 是 Camunda 8 的轻量级发行版, 专为开发和测试环境设计, 无需复杂的容器化部署即可快速启动流程引擎。

1. **下载 Camunda Run 8** 在<https://developers.camunda.com/install-camunda-8/>, 点击“Download the example project”下载示例项目压缩包, 其中包含了 Camunda Run。或访问 Camunda 官方下载页面 (<https://developers.camunda.com/install-camunda-8/>), 下载 Camunda Run 8.8.11 版本。选择 ZIP 压缩包格式。
2. **解压安装**

将下载的 ZIP 包解压到本地目录, 例如:

 - Windows: `C:\camunda\camunda-run`
 - macOS/Linux: `/opt/camunda/camunda-run`
3. **启动 Camunda Run**

打开命令行终端, 进入 Camunda Run 的根目录, 执行启动命令:

- **Windows:** `c8run.exe start`
- **macOS/Linux:** `./c8run.sh start`

启动成功后，控制台将显示类似以下信息：

```
Camunda Run 8.8.11 started successfully
Zeebe Gateway started on port 26500
Operate started on http://localhost:8080/operate
Tasklist started on http://localhost:8080/tasklist
```

一般情况下，会自动弹出浏览器 camunda 登录界面（图 2.1），缺省用户名和密码为：demo/demo。

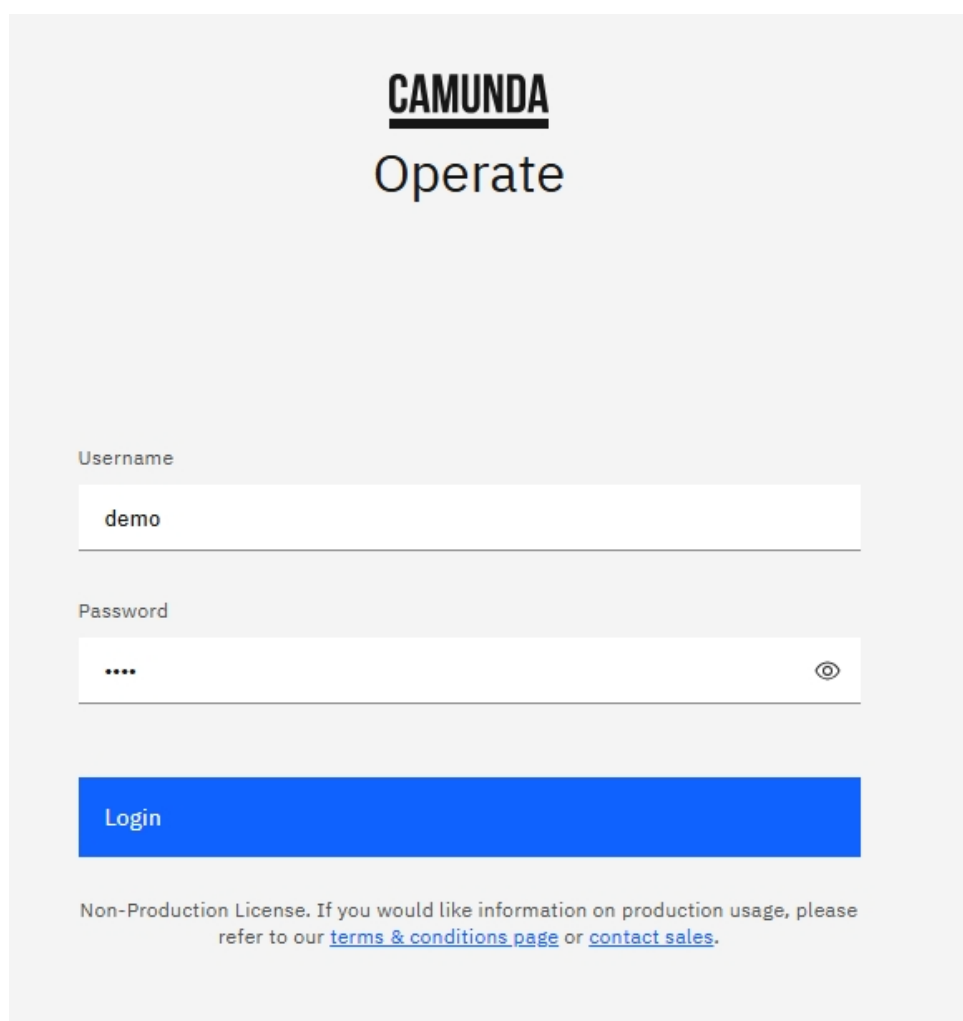


图 2.1: Camunda 登录界面

进入 Camunda Web 界面，可验证 Camunda Run 各组件正常运行（图 2.2）：

- Operate（流程监控）：`http://localhost:8080/operate`
- Tasklist（任务管理）：`http://localhost:8080/tasklist`
- Zeebe Gateway: `localhost:26500`（gRPC 接口，需客户端访问）

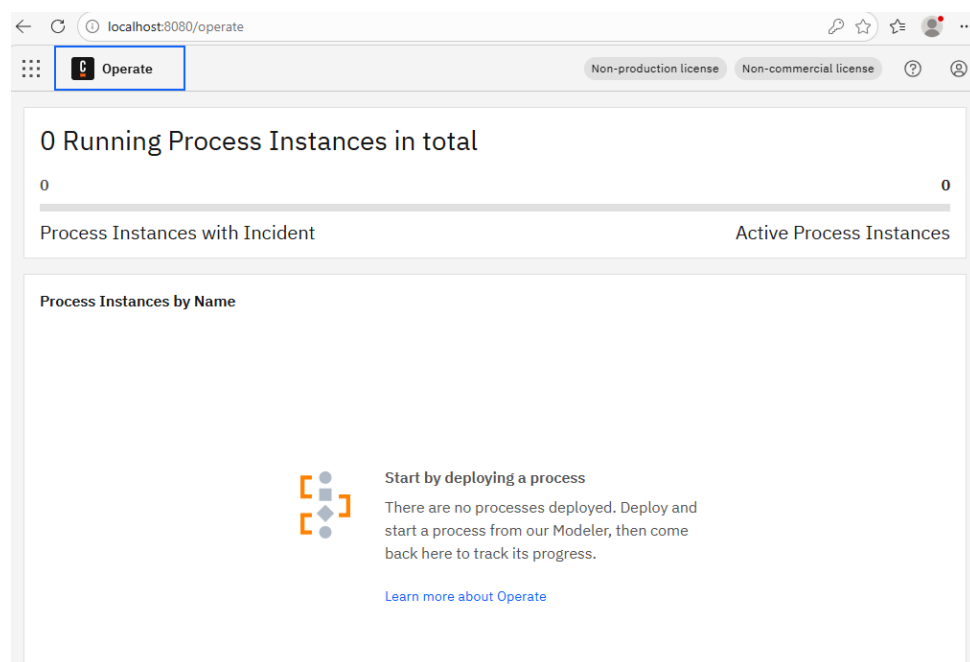


图 2.2: Camunda Operate 监控界面

如需修改 Camunda Run 的默认配置，编辑 `configuration` 目录下的配置文件：

- `application.yml`: 主配置文件，可修改端口号、数据存储等
- `zeebe.cfg.toml`: Zeebe 引擎配置
- `logback.xml`: 日志配置

2.3.4 Camunda Desktop Modeler 安装与配置

Camunda Desktop Modeler 是用于设计 BPMN 流程模型的桌面应用程序，支持本地建模并可直接部署到 Camunda Run。

1. **下载 Camunda Modeler** 在 <https://developers.camunda.com/install-camunda-8/>，点击“Download the example project”下载示例项目压缩包，其中包含了 Camunda Modeler 的安装包。或访问 Camunda 官方网站 (<https://camunda.com/download/modeler/>)，下载适用于操作系统的 Camunda Modeler 5.44.0 版本。
2. **安装 Modeler**
 - **Windows**: 运行下载的 EXE 安装程序，按提示完成安装
 - **macOS**: 打开下载的 DMG 文件，将 Camunda Modeler 拖入 Applications 文件夹
 - **Linux**: 解压下载的 AppImage 文件，赋予执行权限：

```
chmod +x camunda-modeler-5.44.0.AppImage
./camunda-modeler-5.44.0.AppImage
```

3. **配置与 Camunda Run 的连接**

启动 Camunda Modeler 后，需要配置与 Camunda Run 的连接以部署流程模型。

Modeler 通过 Zeebe Gateway 与 Camunda 8 Run 建立连接，具体配置步骤如下：

- 点击左下角的连接状态指示器（或使用快捷键 **Ctrl+,**）打开连接管理器
- 点击“Add connection”，选择目标类型为“Camunda 8 Self-Managed”
- 按表2.3配置连接参数

表 2.3: Camunda Modeler 连接配置参数

字段	填写内容	说明
Cluster URL	<code>http://localhost:26500</code>	Zeebe Gateway 地址，用于部署 BPMN 流程和创建流程实例（必需）
Operate URL	<code>http://localhost:8080/operate</code>	（可选）Operate Web 界面地址，用于从 Modeler 直接打开流程监控视图
Authentication	None	本地开发环境无需认证

配置完成后，Modeler 会自动验证连接状态。连接成功后，出现图 2.3中红圈的成功状态。

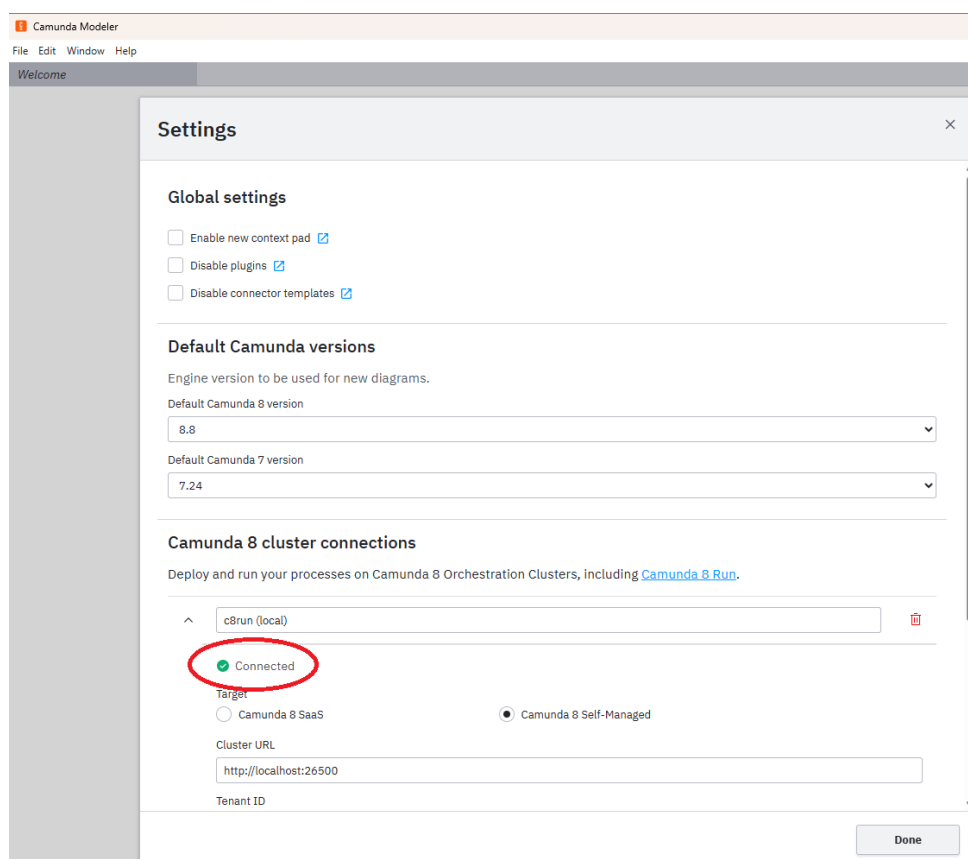


图 2.3: 设置 Camunda Modeler 以与 Camunda 建立连接

2.3.5 Camunda 8 可用性测试

通过两个简单业务流程验证 Camunda Desktop Modeler 与 Camunda 8 Run 的完整性。

第一个测试案例来自 Camunda 官方 getting start 教程。

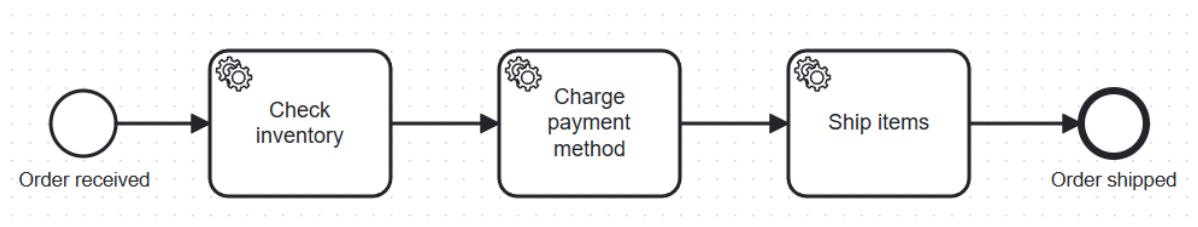


图 2.4: 测试用业务流程模型

Windows 11 + VS Code 操作步骤

1. 下载官方 **getting start** 教程项目 <https://developers.camunda.com/install-camunda-8/>, 点击 “Download the example project” 下载示例项目压缩包, 并解压到本地目录
2. 按官方教程步骤完成业务流程的部署与运行 <https://docs.camunda.io/docs/guides/getting-started-example>
 - (a) 解压下载的 Camunda 8 starter package 文件
 - (b) 使用根目录的 `camunda-start.bat` 启动 Camunda Run
 - (c) 打开文件夹 Camunda Modeler 并启动业务流程模型编辑器, 并打开 `camunda-8-get-started/bpmn/diagram/1.bpmn`
 - (d) 点击下方火箭按钮部署该模型
 - (e) 点击火箭右方的三角按钮, 输入参数 `"item": "special widget"`, 运行一个流程实例,
 - (f) 在 Camunda Operate 界面查看流程实例状态 (<http://localhost:8080/operate/>) (账户和密码均为 demo)
 - (g) 打开 vscode 终端进入 `camunda-8-get-started/nodejs` 目录
 - (h) 安装 Node.js 依赖: `npm install`
 - (i) 启动 Worker: `npm start` (可能要按照 `ts-node` 依赖)
 - (j) 1 在 Worker 控制台查看任务执行结果

3. 故障排查


```
PS D:\camunda8run\start\camunda-8-get-started\nodejs> npm start

> nodejs@1.0.0 start
> npx ts-node ./source/index.ts

Job workers started. Waiting for jobs...

info: Processing check-inventory job: {"timestamp":"2026-03-22T04:04:56.017Z"}
info: Checking inventory for item: special widget {"timestamp":"2026-03-22T04:04:56.017Z"}
info: check-inventory job completed: 2251799813703715 {"timestamp":"2026-03-22T04:04:58.030Z"}
info: Processing charge-payment job: {"timestamp":"2026-03-22T04:04:58.043Z"}
info: charge-payment job completed: 2251799813704388 {"timestamp":"2026-03-22T04:05:00.048Z"}
info: Processing ship-items job: {"timestamp":"2026-03-22T04:05:00.059Z"}
info: ship-items job completed: 2251799813704397 {"timestamp":"2026-03-22T04:05:02.067Z"}
```

图 2.5: vsCode 终端启动 Worker 运行结果

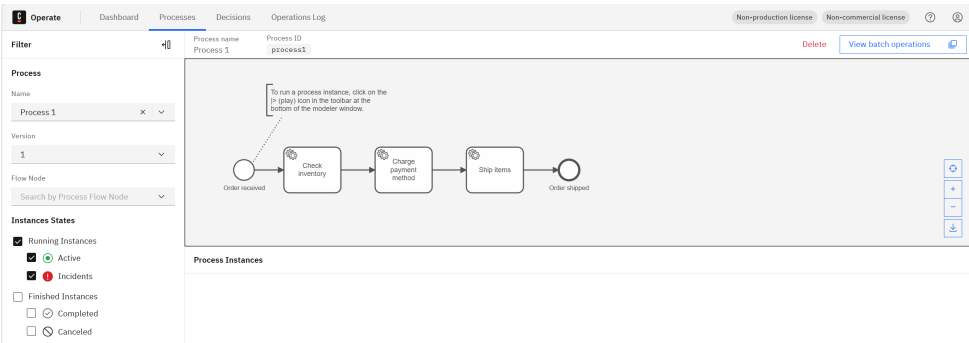


图 2.6: Camunda Operate 界面查看流程实例状态

表 2.4: 常见问题及解决方法

问题	可能原因	解决方法
部署失败	Camunda 8 Run 未启动	检查 http://localhost:26500
Worker 未收到任务	任务类型不匹配	检查 BPMN 中的 task type
Operate 无法访问	服务未完全启动	等待几秒后刷新

第二个测试案例是一个简单的协作业务流程，包含了两个独立的业务流程和它们之间发送并接收的一条消息。

- 按教程步骤完成两个业务流程的部署与运行
 - 使用根目录的 `camunda-start.bat` 启动 Camunda Run
 - 打开文件夹 Camunda Modeler 并启动业务流程模型编辑器，并打开 `message-demo/bpmn/responder-process.bpmn`
 - 点击下方火箭按钮部署该模型
 - 点击火箭右方的三角按钮，输入参数 `"businessKey": "demo-1001", "receiver-Name": "Responder-B"`，运行一个流程实例
 - 打开文件夹 Camunda Modeler 并启动业务流程模型编辑器，并打开 `message-`

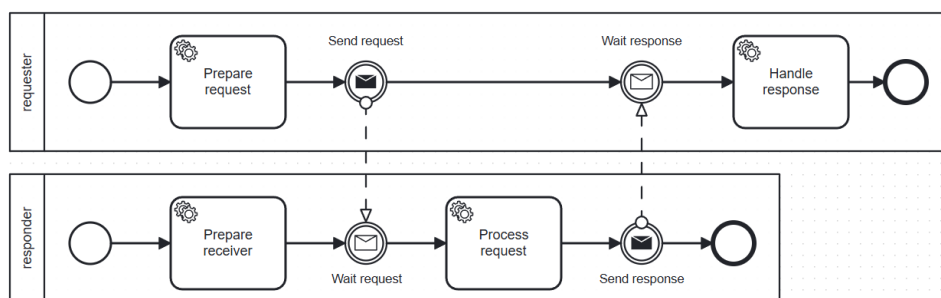


图 2.7: 协作流程示意图

demo/bpmn/requester-process.bpmn

- (f) 点击下方火箭按钮部署该模型
- (g) 点击火箭右方的三角按钮,输入参数 "businessKey": "demo-1001", "customerId": "CUST-001", "senderName": "Requester-A", "requestText": "Please review order CUST-001", 运行一个流程实例
- (h) 在 Camunda Operate 界面查看流程实例状态 (<http://localhost:8080/operate/>) (账户和密码均为 demo)
- (i) 打开 vscode 终端 (ctrl+) 进入 message-demo/nodejs 目录
- (j) 安装 Node.js 依赖: `npm install`
- (k) 启动 Worker: `npm start` (可能要安装 ts-node 依赖)

```

PS C:\Users\Administrator\Desktop\Project\code-camunda\message-demo\nodejs> npm start

> nodejs@1.0.0 start
> npx ts-node ./source/index.ts

Camunda message-pair demo workers started.
REST endpoint: http://localhost:8080
Workers registered:
- prepare-request
- publish-request-message
- handle-response
- prepare-receiver
- process-request
- publish-response-message
Waiting for jobs...

[info] [prepare-receiver] jobKey=2251799813792947 {"timestamp":"2026-03-30T16:23:34.632Z"}
[info] [prepare-receiver] businessKey=demo-1001 {"timestamp":"2026-03-30T16:23:34.633Z"}
[info] [prepare-receiver] receiverName=Responder-B {"timestamp":"2026-03-30T16:23:34.633Z"}
[info] [prepare-receiver] responder process is ready and will wait for the request message. {"timestamp":"2026-03-30T16:23:34.634Z"}
[info] [prepare-request] jobKey=2251799813793045 {"timestamp":"2026-03-30T16:23:46.012Z"}
[info] [prepare-request] businessKey=demo-1001 {"timestamp":"2026-03-30T16:23:46.012Z"}
[info] [prepare-request] customerId=CUST-001 {"timestamp":"2026-03-30T16:23:46.012Z"}
[info] [prepare-request] senderName=Requester-A {"timestamp":"2026-03-30T16:23:46.012Z"}
[info] [prepare-request] requestText=Please review order CUST-001 {"timestamp":"2026-03-30T16:23:46.013Z"}
[info] [publish-request-message] jobKey=2251799813793051 {"timestamp":"2026-03-30T16:23:46.027Z"}
[info] [publish-request-message] payload={"name":"demo-request-message","correlationKey":"demo-1001","variables":{"businessKey":"demo-1001","customerId":"CUST-001","senderName":"Requester-A","requestText":"Please review order CUST-001","requestSentAt":"2026-03-30T16:23:46.027Z","fromProcess":"requester-process"}} {"timestamp":"2026-03-30T16:23:46.028Z"}
[info] [publish-request-message] response={"tenantId":"<default>","messageKey":"2251799813793054","processInstanceKey":"2251799813792940"} {"timestamp":"2026-03-30T16:23:46.067Z"}
[info] [process-request] jobKey=2251799813793063 {"timestamp":"2026-03-30T16:23:46.076Z"}
[info] [process-request] businessKey=demo-1001 {"timestamp":"2026-03-30T16:23:46.077Z"}
[info] [process-request] senderName=Requester-A {"timestamp":"2026-03-30T16:23:46.077Z"}
[info] [process-request] requestText=Please review order CUST-001 {"timestamp":"2026-03-30T16:23:46.077Z"}
[info] [process-request] receiverName=Responder-B {"timestamp":"2026-03-30T16:23:46.078Z"}
[info] [process-request] responseText=Approved by Responder-B: Please review order CUST-001 {"timestamp":"2026-03-30T16:23:46.078Z"}
[info] [process-request] allVariables={"fromProcess":"requester-process","requestText":"Please review order CUST-001","senderName":"Requester-A","receiverReadyAt":"2026-03-30T16:23:46.634Z","receiverName":"Responder-B","customerId":"CUST-001","businessKey":"demo-1001","requestSentAt":"2026-03-30T16:23:46.027Z"} {"timestamp":"2026-03-30T16:23:46.078Z"}
[info] [publish-response-message] jobKey=2251799813793078 {"timestamp":"2026-03-30T16:23:46.097Z"}
[info] [publish-response-message] payload={"name":"demo-response-message","correlationKey":"demo-1001","variables":{"businessKey":"demo-1001","receiverName":"Responder-B","response":
  
```

图 2.8: vsCode 终端启动 Worker 运行结果

2. 故障排查

验证要点

这两个测试案例验证了以下功能:

- Camunda Modeler 与 Zeebe Gateway 的连接
- BPMN 流程的部署
- 流程实例的创建和启动
- 服务任务的调度和执行
- 流程实例的正常结束
- 消息事件的发送和接收
- Camunda Operate 的流程监控功能

2.3.6 CPN Tools 安装

官网明确“CPN Tools 已被 CPN IDE 替代”，所以建议优先装 CPN IDE（下一小节），不建议再装 CPN Tools。

CPN Tools 是用于着色 Petri 网建模与仿真的专业工具，用于验证跨组织协作流程的并发性质。

1. 下载 CPN Tools

访问 CPN Tools 官方网站 (<http://cpntools.org/download/>)，下载适用于你操作系统的版本：

- Windows: cpntools-4.0.1-windows.exe
- Linux: cpntools-4.0.1-linux.tar.gz
- macOS: cpntools-4.0.1-macos.dmg

2. 安装 CPN Tools

- **Windows:** 运行下载的 EXE 安装程序，按提示完成安装。建议安装路径为 C:\Program Files\CPN Tools。
- **macOS:** 打开下载的 DMG 文件，将 CPN Tools 拖入 Applications 文件夹。首次运行时，需要在“系统偏好设置 安全性与隐私”中允许运行。
- **Linux:**

```

1 tar -xzf cpntools-4.0.1-linux.tar.gz
2 sudo mv cpntools /opt/
3 sudo apt-get update
4 sudo apt-get install -y libgtk2.0-0 libglade2-0
   libgnomecanvas2-0
5 echo 'export PATH=$PATH:/opt/cpntools' >> ~/.bashrc
6 source ~/.bashrc
7

```

3. 验证安装

启动 CPN Tools，应看到类似以下界面：

4. 创建并运行一个简单的 Petri 网模型

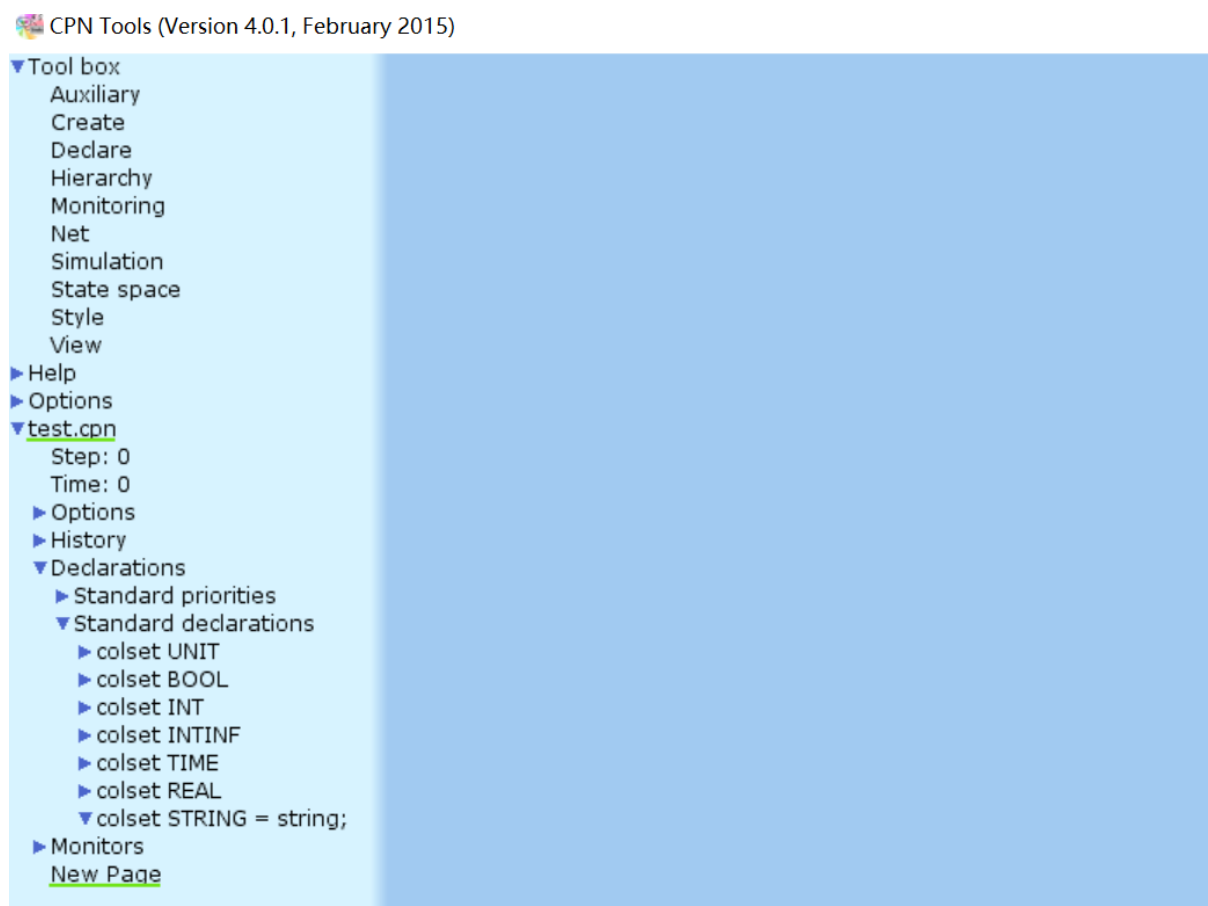


图 2.9: CPN Tools 主界面

为了验证 CPN Tools 的功能，创建一个简单的通信模型（可以直接进入 cpntools 后右键空白处 load net 选择 cpntools 文件夹中的 test.cpn 来加载，直接跳到第 i 步）：

- (a) 在 CPN Tools 中点击"File New"（在右侧空白处右键按住不动选择 new net）
- (b) 创建两个 Place：分别命名为"Request"和"Response"（在新的 page 中右键选择 new place，单击可以改名）
- (c) 创建一个 Transition：命名为"Process Request"（在新的 page 中右键选择 new transition，单击可以改名）
- (d) 用 Arc 连接 Request Process Request Response（在 place/transition 上右键选择 new arc 然后点击要连接的元素，不想链接了可以右键选择 drop tool）
- (e) 设置颜色集：创建新颜色集 `colset STRING = string;`（左侧菜单中默认已有：Declarations/Standard declarations，无需操作）
- (f) 设置 Place 类型：将两个 Place 的颜色集都设为 STRING（左键对应的 Place，单击 Tab，输入 STRING 后回车）
- (g) 在 Request Place 中初始化一个 Token：1 "ManifestRequest"（单击 place，点击两次 Tab，出现 INIT MARK 后输入 1 "ManifestRequest"）
- (h) 对 Request Process Request 这条边增加 text 为 1 "ManifestRequest"，对

Process Request Response 这条边增加 text 为 1`"ManifestResponse"

- (i) 点击"Simulation" 菜单, 选择第四个按钮 “execute a transition”, 点击 Process Request, 观察 Token 的流动

CPN Tools (Version 4.0.1, February 2015)

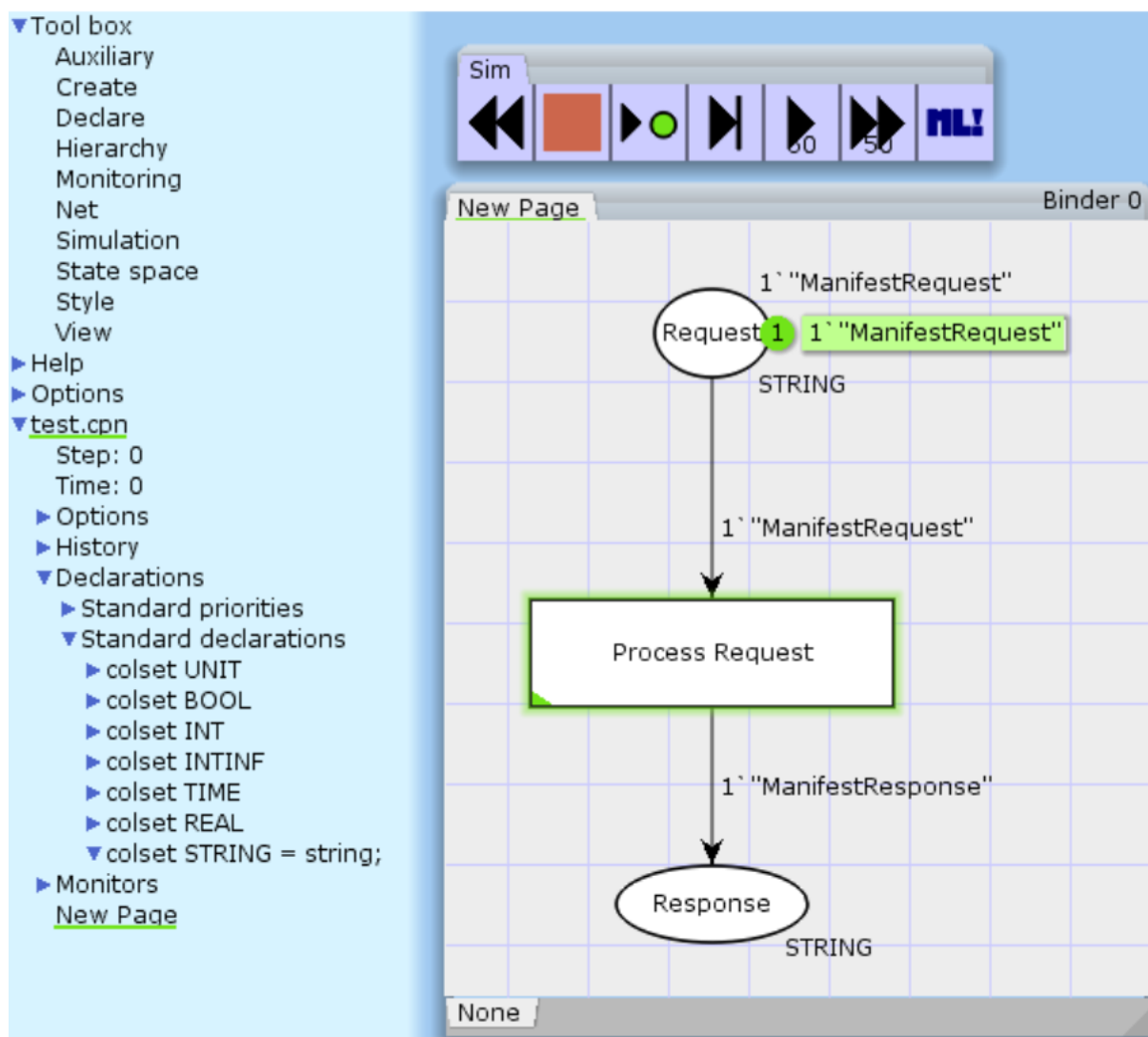


图 2.10: CPN Tools 主界面
[CPN Tools 中创建的简单模型截图]

5. 检验方法

成功标准:

- CPN Tools 能够正常启动, 无错误提示
- 能够创建新的 Petri 网模型
- 能够设置颜色集和 Place 类型
- 能够初始化 Token 并执行仿真
- 仿真过程中 Token 能够按照预期流动

2.3.7 CPN IDE 安装

1. 下载 CPN IDE

访问 CPN IDE 官方网站下载界面 (<https://cpnide.org/latest-downloads/>), 下载 Windows 版本:

2. 准备 Java 环境 CPN IDE 基于 Java 开发, 确保已安装 Java 17+ 版本, 并配置好环境变量 (参见前面 Java 开发环境配置部分)。

3. 安装 CPN IDE

安装完, 打开 CPN IDE, 界面如下图所示:

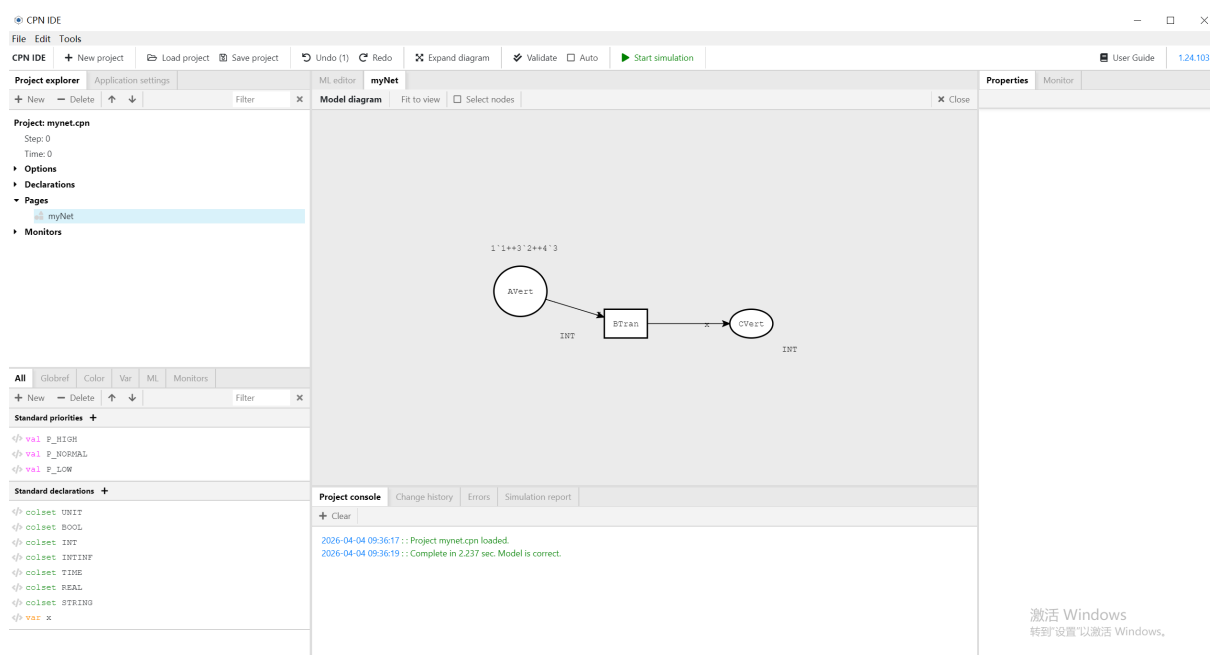


图 2.11: CPN IDE 主界面
[CPNIDE]

4. 创建并运行一个简单的 Petri 网模型

- 在 CPN IDE 主界面中右键删去中间区域的 Place 和 Transition
- 在中间区域右键创建两个 Place: 分别命名为"Request" 和"Response"
- 在中间区域右键创建一个 Transition: 命名为"Process Request"
- 右键 Place/Transition 用 Arc 连接 Request Process Request Response
- 设置颜色集:在左下方的 Declaration 创建新颜色集 `colset STRING = string;` (默认已有, 无需操作)
- 设置 Request Place 的初始 Token: 单击 Request Place, 将右侧栏 Properties-Initial Marking 的设为 `1*"ManifestRequest"`
- 单击 Request Process Request 这条边, 在右侧栏 Connection-Annotation 中设置为 `1*"ManifestRequest"`
- 单击 Process Request Response 这条边, 在右侧栏 Connection-Annotation

中设置为 1 "ManifestResponse"

- (i) 点击顶部菜单栏的 "Start Simulation" 按钮，点击已使能的 Transition，观察 Token 的流动

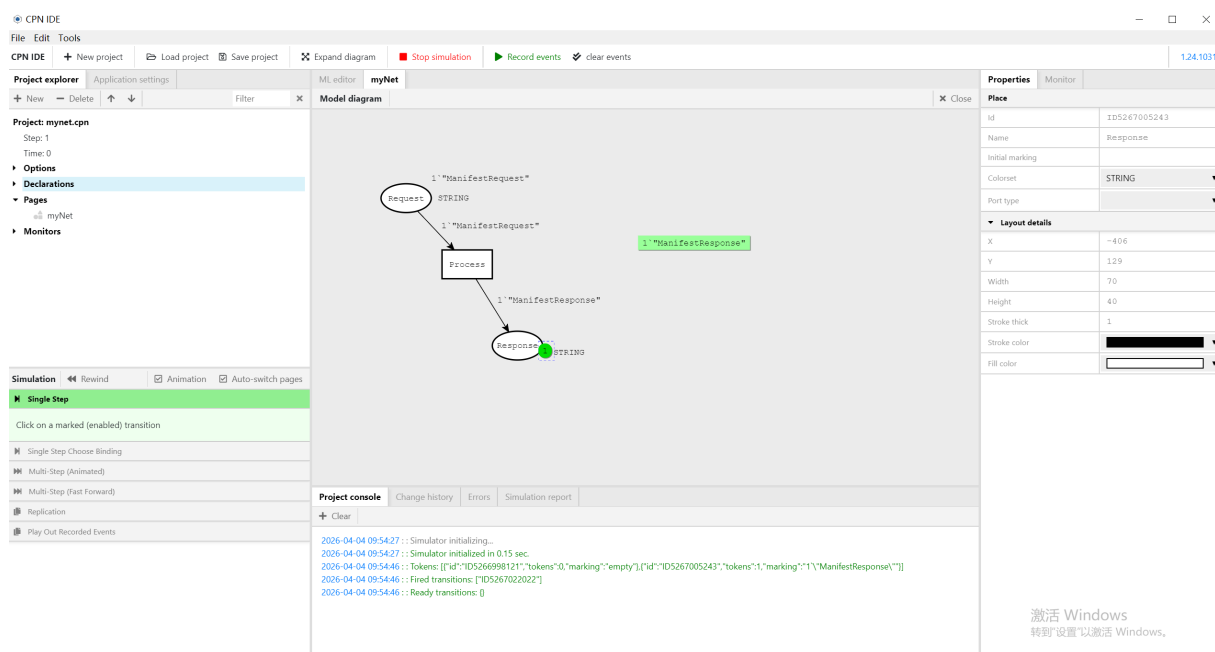


图 2.12: CPN IDE 的 Simulation

2.3.8 mCRL2 安装

mCRL2 是基于进程代数的形式化验证工具，用于分析分布式系统的行为一致性。

1. 下载 mCRL2

访问 mCRL2 官方网站 (https://www.mcrl2.org/web/user_manual/download.html)，下载适用于你操作系统的版本：

- Windows: mcrl2-win64-202311.7z
- macOS: mcrl2-macos-202311.dmg
- Linux: mcrl2-ubuntu-22.04-202311.deb 或源码编译

2. 安装 mCRL2

- **Windows:** 解压下载的 7z 文件到指定目录，如 C:\Program Files\mcrl2。将 bin 目录添加到系统 PATH 环境变量。
- **macOS:** 打开下载的 DMG 文件，将 mCRL2 拖入 Applications 文件夹。
- **Linux (Ubuntu/Debian):**

```
1 sudo dpkg -i mcrl2-ubuntu-22.04-202311.deb
2 # 如遇依赖问题，执行
3 sudo apt-get install -f
4
```

其他 Linux 发行版可从源码编译:

```
1 git clone https://github.com/mCRL2org/mCRL2.git
2 cd mCRL2
3 mkdir build && cd build
4 cmake .. -DCMAKE_INSTALL_PREFIX=/usr/local
5 make -j4
6 sudo make install
7
```

3. 验证安装

打开命令行终端, 执行以下命令验证 mCRL2 安装:

```
1 mcrl22lps --version
2 lps2pbes --version
3 pbes2bool --version
4
```

预期输出应显示版本号为 202311。

4. 创建并验证一个简单的 mCRL2 模型

创建一个简单的"请求-响应"模型文件 `simple.mcrl2`(可以直接使用 mCRL2/test 工程内的文件, 后续操作也全部可以用 mCRL2 IDE 完成):

```
1 % Simple request-response model
2 act
3     send_request,recv_request,send_response,recv_response,
4     request,response;
5
6 proc
7     Requester = send_request . recv_response . Requester;
8     Responder = recv_request . send_response . Responder;
9
10 init
11     allow({request, response},
12         comm({
13             send_request | recv_request -> request,
14             send_response | recv_response -> response
15         },
16         Requester || Responder
17     ));
18
```


执行以下命令验证模型：

```

1 # 生成线性进程
2 mcr122lps simple.mcr12 simple.lps
3
4 # 生成状态空间
5 lps2lts simple.lps simple.aut
6
7 # 检查死锁
8 lps2pbes -f deadlock.mcf simple.lps deadlock.pbes
9 pbes2bool deadlock.pbes
10

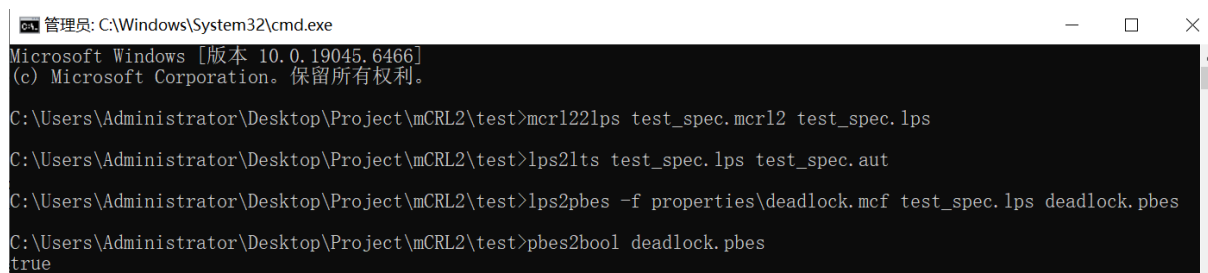
```

其中 deadlock.mcf 文件内容为：

```

1 % Deadlock freedom property
2 [ true* ] < true > true
3

```



```

管理员: C:\Windows\System32\cmd.exe
Microsoft Windows [版本 10.0.19045.6466]
(c) Microsoft Corporation。保留所有权利。

C:\Users\Administrator\Desktop\Project\mCRL2\test>mcr122lps test_spec.mcr12 test_spec.lps

C:\Users\Administrator\Desktop\Project\mCRL2\test>lps2lts test_spec.lps test_spec.aut

C:\Users\Administrator\Desktop\Project\mCRL2\test>lps2pbes -f properties\deadlock.mcf test_spec.lps deadlock.pbes

C:\Users\Administrator\Desktop\Project\mCRL2\test>pbes2bool deadlock.pbes
true

```

图 2.13: mCRL2 命令行运行截图
[mCRL2 命令行运行截图]

5. 检验方法

成功标准：

- mcr122lps、lps2pbes、pbes2bool 等命令能够正常执行并显示版本信息
- 能够成功编译简单的 mCRL2 模型
- 能够生成线性进程和状态空间
- 能够验证简单的模态 - 演算性质

2.3.9 KeYmaera X 安装与 Mathematica 配置

KeYmaera X 是用于混合系统验证的定理证明器，基于微分动态逻辑。Mathematica 可增强其算术求解能力。

1. 下载 KeYmaera X

访问 KeYmaera X 官方网站 (<https://keymaerax.org>)，下载最新版本（推荐 4.9.2）：

- 跨平台 JAR 版本: `keymaerax-4.9.2.jar`
- 各操作系统安装包: 根据系统选择 EXE、DMG 或 DEB 文件

2. 安装 KeYmaera X

- 跨平台 JAR 版本（需要 Java 环境）：

```
1 java -jar keymaerax-4.9.2.jar
```

```
2
```

首次运行会提示选择工作空间目录,建议创建专用目录如 `C:\keymaerax-workspace`。

- **Windows:** 运行下载的 EXE 安装程序,按提示完成安装。
- **macOS:** 打开下载的 DMG 文件,将 KeYmaera X 拖入 Applications 文件夹。
- **Linux:** 对于 DEB 包:

```
1 sudo dpkg -i keymaerax-4.9.2.deb
```

```
2
```

3. 首次启动与配置

首次启动 KeYmaera X 会进入配置向导：

- 选择 Z3 定理证明器（内置）或配置 Mathematica（如已安装）
- 选择工作空间目录
- 等待初始化完成

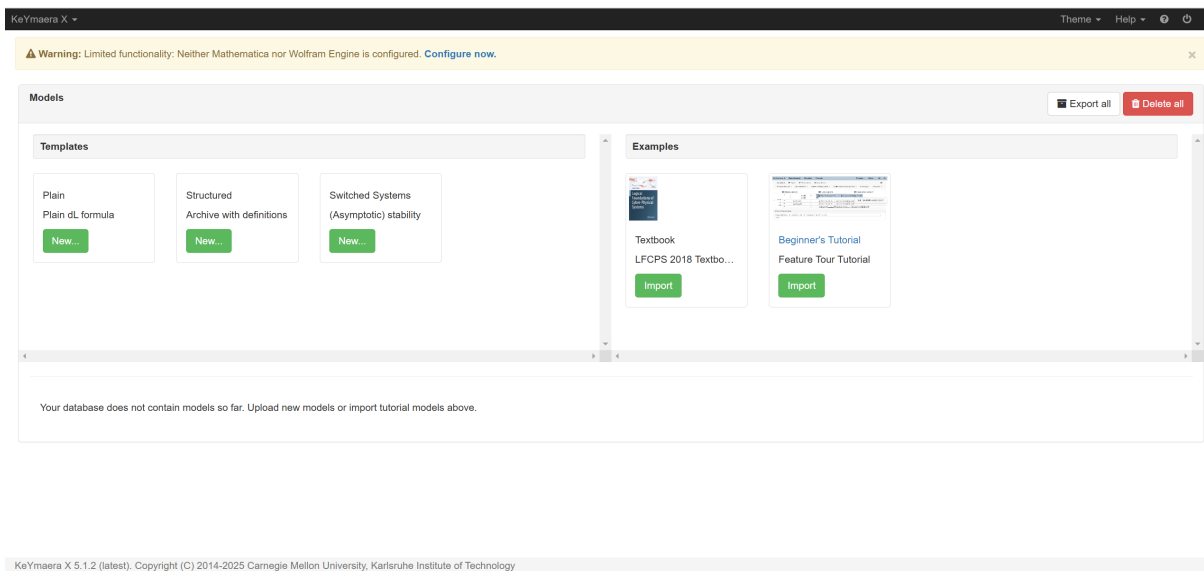


图 2.14: KeYmaera X 启动界面截图

4. 安装 Mathematica（可选，用于增强算术求解）

Mathematica 可显著提升 KeYmaera X 的算术求解能力，特别是处理复杂微分方程时。如无许可证，可跳过此步，KeYmaera X 将使用内置的 Z3 求解器。

- 访问 Wolfram 官方网站 (<https://www.wolfram.com/mathematica/>)，下载 Mathematica 13.0+ 版本

(b) 安装 Mathematica (具体步骤略, 按安装向导操作)

(c) 获取许可证 (可通过教育机构获取免费许可)

5. 配置 KeYmaera X 与 Mathematica 集成

如已安装 Mathematica, 需要在 KeYmaera X 中配置连接:

(a) 启动 KeYmaera X

(b) 点击菜单栏 "File Preferences"

(c) 在 "MathKernel" 选项卡中, 设置 Mathematica 内核路径:

- Windows: C:\Program Files\Wolfram Research\Mathematica\13.0\MathKernel.exe
- macOS: /Applications/Mathematica.app/Contents/MacOS/MathKernel
- Linux: /usr/local/Wolfram/Mathematica/13.0/Executables/MathKernel

(d) 点击 "Test Connection" 验证连接

(e) 保存配置

6. 验证一个简单的混合系统模型例子

为了验证 KeYmaera X 的功能, 创建一个简单的 AGV 运动模型:

(a) 在 Examples 中点击 "Beginner's Tutorial" 的 import

(b) 在下方点击 "Beginner Safety Tutorial" - "00:Forward-Driving Car", 加载模型

(c) 点击右上角 load proof, 观察证明过程和结果

(d) 点击工具栏的 "Prove" 按钮开始证明

(e) 观察证明过程和结果

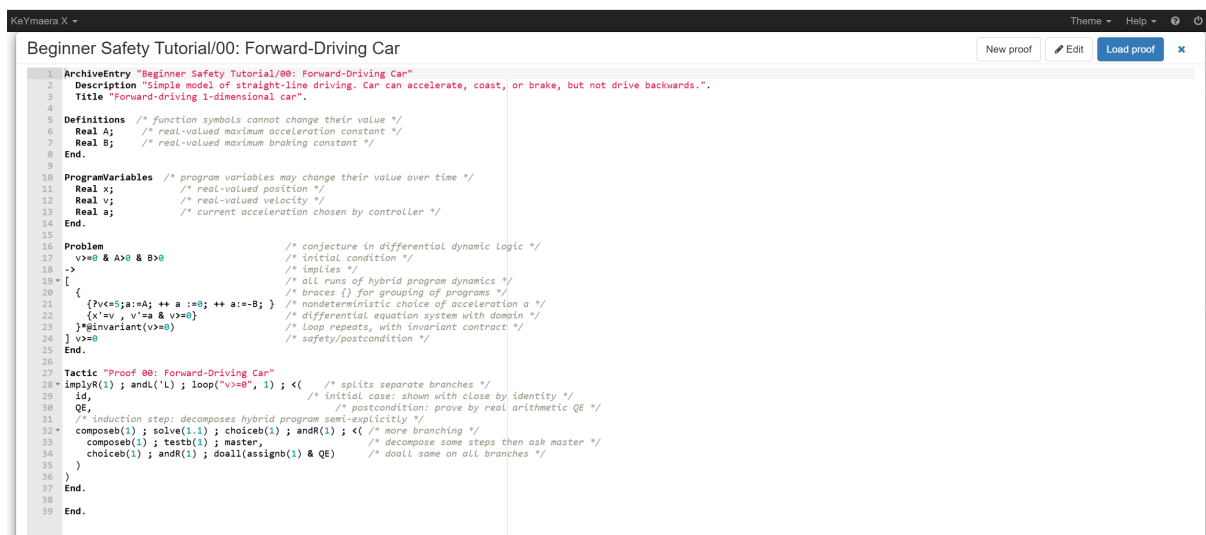


图 2.15: KeYmaera X 模型编辑
[KeYmaera X 模型编辑]

7. 检验方法

成功标准:

- KeYmaera X 能够正常启动, 无错误提示

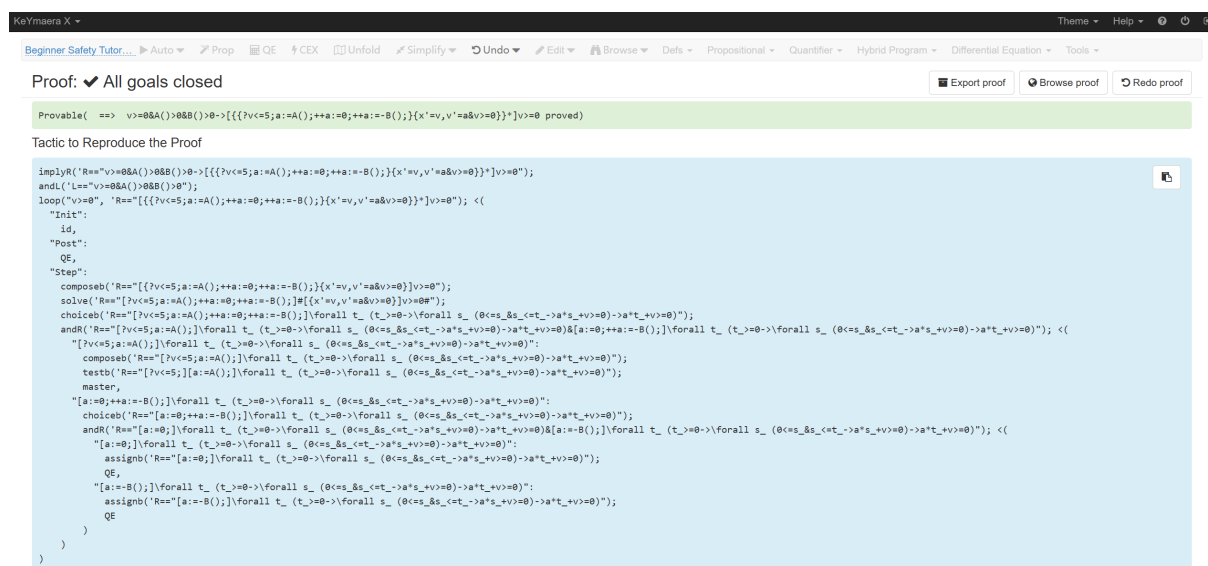


图 2.16: KeYmaera X 证明界面截图

[KeYmaera X 证明界面截图]

- 能够创建新的模型文件
- 能够对简单模型执行自动证明（或至少启动证明过程）
- 如配置了 Mathematica，连接测试应成功
- 证明过程中能够应用微分动态逻辑规则

2.3.10 辅助工具安装

为完整实验环境，还需要安装以下辅助工具：

1. Docker Desktop（可选）

用于容器化部署和测试，非必需但推荐：

- 访问 <https://www.docker.com/products/docker-desktop> 下载
- 按安装向导完成安装
- 验证安装: `docker -version`

2. Git

用于版本控制和代码管理：

- 访问 <https://git-scm.com/downloads> 下载
- 按安装向导完成安装
- 验证安装: `git -version`

3. Visual Studio Code

用于编辑代码和实验文档：

- 访问 <https://code.visualstudio.com> 下载
- 按安装向导完成安装
- 推荐安装插件：LaTeX Workshop、Java Extension Pack、BPMN Editor

4. LaTeX 发行版

用于编译实验报告：

- Windows: 安装 MiKTeX (<https://miktex.org>)
- macOS: 安装 MacTeX (<https://tug.org/mactex/>)
- Linux: 安装 TeX Live (`sudo apt-get install texlive-full`)

2.4 预期结果

完成本章所有实验步骤后，应达到以下预期结果：

1. **Java 环境：** OpenJDK 21 正确安装，`java -version` 显示正确版本信息。
2. **Camunda 环境：**
 - Camunda Run 8 能够正常启动，Operate (<http://localhost:8081>) 和 Tasklist (<http://localhost:8082>) 可访问
 - Camunda Modeler 能够正常启动，并与 Camunda Run 成功建立连接
 - 能够通过 Modeler 设计简单流程并成功部署到 Camunda Run
 - 能够在 Operate 中查看已部署的流程定义和运行中的流程实例
3. **CPN Tools 环境：**
 - CPN Tools 能够正常启动
 - 能够创建简单的 Petri 网模型并执行仿真
4. **mCRL2 环境：**
 - 所有 mCRL2 命令行工具 (`mcr122lps`、`lps2pbcs`、`pbcs2bool` 等) 可正常执行
 - 能够编译简单的 mCRL2 模型并验证性质
5. **KeYmaera X 环境：**
 - KeYmaera X 能够正常启动
 - 能够创建简单的混合系统模型并启动证明
 - (可选) Mathematica 配置成功，连接测试通过
6. **辅助工具：** 所有辅助工具 (Docker、Git、VS Code、LaTeX) 安装成功，版本命令可正常执行。

2.5 检验方法

2.5.1 环境整体检验脚本

创建一个简单的检验脚本 `verify-environment.sh` (Linux/macOS) 或 `verify-environment.bat` (Windows)，自动检查各工具是否安装成功：

Listing 2.1: 环境检验脚本 (Linux/macOS)

```
1 #!/bin/bash
2
3 echo "=== 跨组织协作实验环境检验 ==="
4 echo ""
5
6 # Java 检验
7 echo -n "Java: "
8 if command -v java &> /dev/null; then
9     echo " $(java -version 2>&1 | head -n 1)"
10 else
11     echo " 未安装"
12 fi
13
14 # Camunda Run 检验
15 echo -n "Camunda Run: "
16 if curl -s http://localhost:8081/actuator/health &> /dev/null;
17     then
18         echo " 运行中 (http://localhost:8081)"
19     else
20         echo " 未运行"
21     fi
22
23 # CPN Tools 检验
24 echo -n "CPN Tools: "
25 if command -v cpntools &> /dev/null || [ -d "/opt/cpntools" ];
26     then
27         echo " 已安装"
28     else
29         echo " 未安装"
30     fi
31
32 # mCRL2 检验
33 echo -n "mCRL2: "
34 if command -v mcrl2lps &> /dev/null; then
35     echo " $(mcrl2lps --version | head -n 1)"
36 else
37     echo " 未安装"
38 fi
39
40 # KeYmaera X 检验
```

```

39 echo -n "KeYmaera X: "
40 if [ -f "/opt/keymaeraX-4.9.2.jar" ] || [ -d "/Applications/
    KeYmaera X.app" ]; then
41     echo " 已安装"
42 else
43     echo " 未安装"
44 fi
45
46 # Docker 检验
47 echo -n "Docker: "
48 if command -v docker &> /dev/null; then
49     echo " $(docker --version)"
50 else
51     echo " 未安装"
52 fi
53
54 # Git 检验
55 echo -n "Git: "
56 if command -v git &> /dev/null; then
57     echo " $(git --version)"
58 else
59     echo " 未安装"
60 fi
61
62 echo ""
63 echo "=== 检验完成 ==="

```

Listing 2.2: 环境检验脚本 (Windows)

```

1 @echo off
2 echo === 跨组织协作实验环境检验 ===
3 echo.
4
5 REM Java 检验
6 echo Java:
7 java -version 2>&1 | find "version" > nul
8 if %errorlevel% equ 0 (
9     java -version 2>&1 | find "version"
10 ) else (
11     echo 未安装

```

```
12 )  
13  
14 REM Camunda Run 检验  
15 echo Camunda Run:  
16 curl -s http://localhost:8081/actuator/health > nul 2>&1  
17 if %errorlevel% equ 0 (  
18     echo    运行中 (http://localhost:8081)  
19 ) else (  
20     echo    未运行  
21 )  
22  
23 REM 其他检验略...
```

2.5.2 集成测试：完整的简单协作流程

为了验证所有工具的协同工作能力，建议执行以下集成测试：

1. 在 Camunda Modeler 中设计一个简单的协作流程，包含货主、货代、船代三个参与方，以及订单发送、舱单请求、舱单响应三个消息交互。
2. 部署到 Camunda Run，并启动一个流程实例。
3. 在 CPN Tools 中构建相同逻辑的着色 Petri 网模型，执行仿真验证并发性质。
4. 在 mCRL2 中构建相同逻辑的进程代数模型，验证消息交互的一致性。
5. 在 KeYmaera X 中构建码头 AGV 运动的简化模型，验证安全性质。
6. 对比各工具的验证结果，分析同一业务逻辑在不同形式化模型中的表现。

通过以上步骤，可以全面检验工具链的安装正确性和协同工作能力。

2.6 常见问题与解决方法

表 2.5: 常见安装问题及解决方法

问题现象	可能原因	解决方法
<code>java -version</code> 命令找不到	JAVA_HOME 未正确设置或 PATH 未包含	检查 JAVA_HOME 环境变量，确保指向正确 JDK 路径；检查 PATH 中是否包含 %JAVA_HOME%\bin
Camunda Run 启动后无法访问 Operate	端口被占用或启动失败	检查端口 8081、8082 是否被其他程序占用；查看启动日志中的错误信息
Camunda Modeler 无法部署流程到 Camunda Run	连接配置错误或 Camunda Run 未运行	检查 Modeler 中 Zeebe Gateway 地址是否为 localhost:26500；确保 Camunda Run 已启动
CPN Tools 启动时提示缺少 GTK 库	Linux 系统缺少图形库	安装 libgtk2.0-0、libglade2-0 等依赖库
mCRL2 命令执行时提示找不到共享库	Linux 系统缺少依赖	安装 libgomp1 等依赖库
KeYmaera X 启动时提示 Java 版本过低	使用了过低的 Java 版本	确保使用 Java 11 或更高版本
KeYmaera X 与 Mathematica 连接失败	Mathematica 内核路径配置错误	确认 Mathematica 已正确安装，检查内核路径是否正确

第 II 部分

业务流程建模

第 3 章 集装箱出口协作流程

3.1 流程概述

一个完整的集装箱出口流程（图 1.1）涉及九个独立组织。该流程包含二十八条不同的消息流和四十四个服务任务。图3.1进一步扩展了九个参与方的部分暴露的内部逻辑。该图展示了跨组织业务流程特有的复杂交互模式。

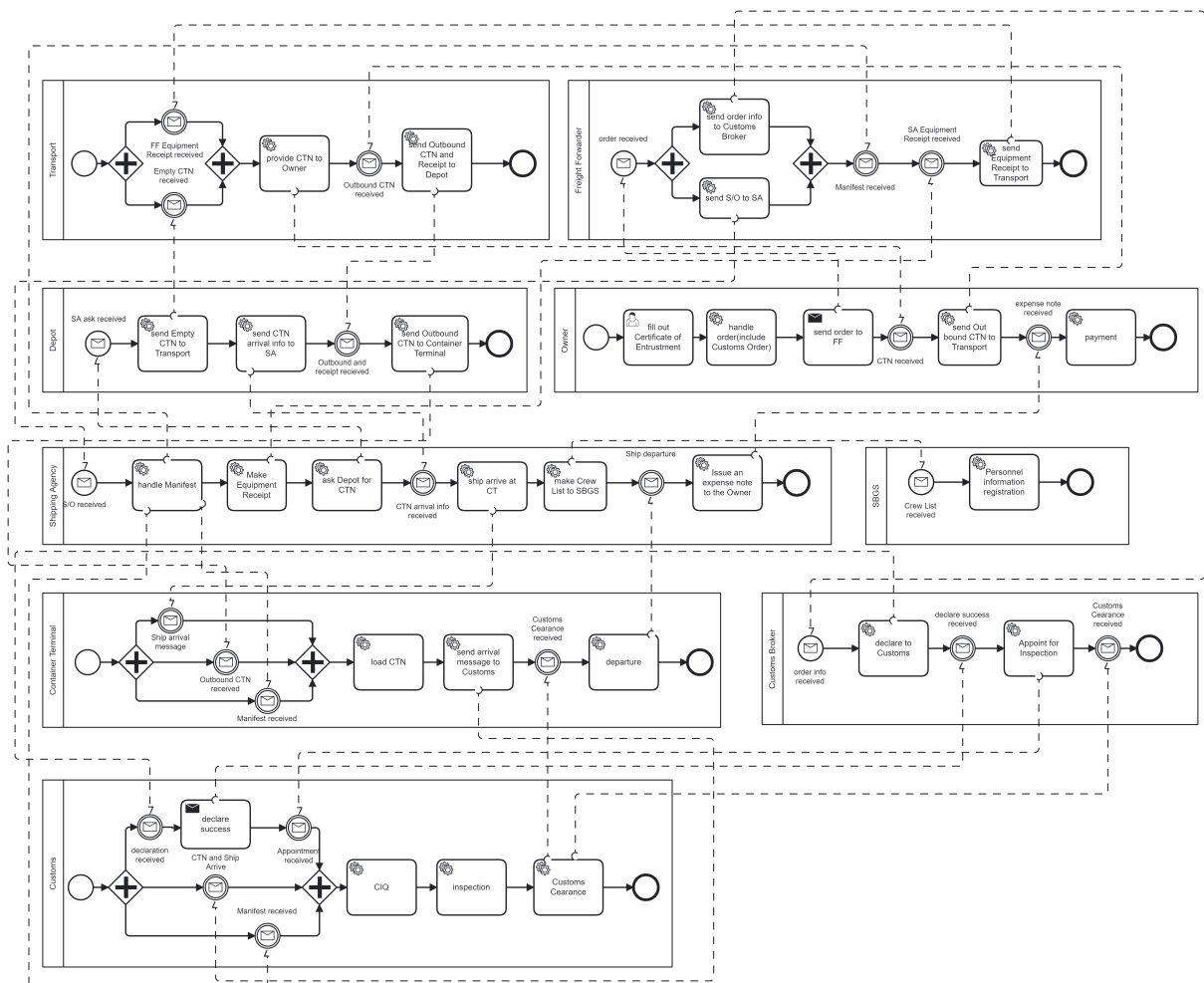


图 3.1: 包含所有 9 个参与方和部分消息流的完整 BPMN 协作图

3.2 完整消息流规范

表3.1列出了协作流程中所有二十八条消息流，指定了每条交互的发送方、接收方和消息类型。

表 3.1: 完整消息流规范

编号	发送方	接收方	消息类型
1	货主	货代	托运订单
2	货代	货主	订单确认
3	货代	船代	舱单请求
4	货代	船代	EIR 请求
5	货代	报关行	报关请求
6	船代	货代	舱单文件
7	船代	货代	EIR 文件
8	船代	码头	船期表
9	船代	边防	船员名单
10	报关行	海关	报关单
11	报关行	货代	报关状态
12	海关	报关行	放行证书
13	海关	报关行	查验指令
14	报关行	海关	查验报告
15	边防	船代	边防放行
16	码头	货代	泊位分配
17	码头	车队	进港预约
18	码头	船代	装船报告
19	码头	海关	船舶离港通知
20	车队	货场	空箱请求
21	货场	车队	空箱发放
22	车队	码头	重箱交付
23	码头	车队	收据确认
24	环境模拟	所有参与方	天气预警
25	环境模拟	码头	高峰负荷预警
26	环境模拟	车队	交通更新
27	环境模拟	海关	随机查验触发
28	环境模拟	所有参与方	系统事件通知

3.3 核心并行汇聚点

在跨组织业务流程中，并行汇聚点（Parallel Synchronization Point）是指多个并行执行的分支必须全部到达后才能继续推进的节点，是工作流模式中的核心概念之一^[8]。

并行汇聚点是跨组织业务流程的核心控制节点，既是业务逻辑的体现，也是技术实现的关键，更是流程优化的着力点。集装箱出口流程中的三个汇聚点，深刻展示了如何通过合理的同步机制，在保持各组织自主性的同时，实现整体流程的协调一致。在集装箱出口流程中，三个关键的并行汇聚点深刻反映了跨组织协作的本质特征和技术挑战。

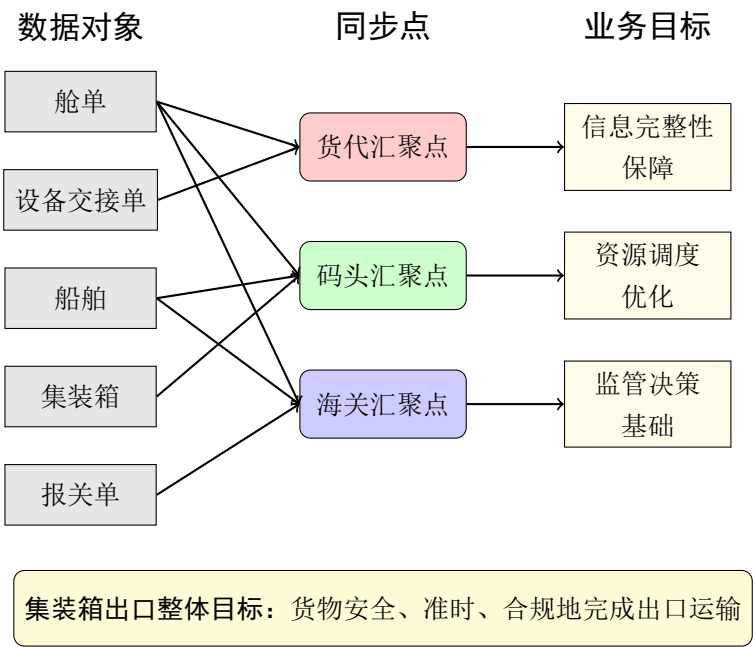


图 3.2: 数据对象、汇聚点与业务目标的关联关系

这三个汇聚点分别位于货代、码头和海关三个关键参与方，共同构成了流程的核心控制节点。

下面将分别详细讨论每个汇聚点的业务意义、技术实现和优化价值。

3.3.1 货代汇聚点：信息完整性的保障

货代汇聚点位于货代接收船代响应的环节，要求同时收到舱单（Manifest）和设备交接单（Equipment Interchange Receipt, EIR）才能继续推进后续操作。

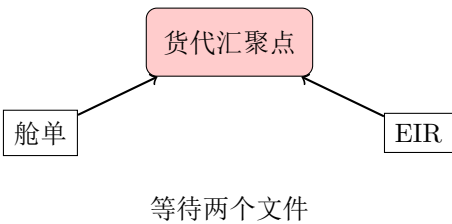


图 3.3: 货代汇聚点：舱单与 EIR 的同步

业务意义

舱单是货物装船的法律依据，记录了货物名称、数量、重量、收货人等信息，是船舶运输的核心文件。设备交接单是集装箱流转的凭证，记录了集装箱号、封号、交接状态等信息，是陆路运输和堆场管理的依据。两者分别从“货物”和“容器”两个维度描述了出口业务，缺一不可。

如果货代在只收到舱单的情况下就安排货物进港，将面临以下风险：

- 集装箱无法在码头合法交接，产生滞港费
- 设备交接单缺失导致提箱和还箱流程混乱
- 船公司可能拒绝装载无 EIR 的集装箱
- 后续的陆路运输和堆场管理无法正常进行

这个汇聚点确保了货代在获得完整信息后才推进流程，避免了因信息不全导致的业务中断和法律风险。

并发控制机制

从技术实现角度看，货代汇聚点需要处理以下并发控制问题：

1. **消息时序不确定性：**舱单和 EIR 可能以任意顺序到达，系统必须能够处理两种顺序，并在第二个消息到达时触发同步。
2. **超时处理：**当某个消息延迟时，系统需要启动补偿机制。实践中常设置 48 小时超时阈值，超时后自动触发：
 - 向船代发送催办通知
 - 通知货主可能延迟
 - 记录服务级别协议违约
3. **错误恢复：**如果舱单和 EIR 内容不一致（如集装箱号不匹配），系统需要暂停流程并触发人工处理。

性能指标

货代汇聚点的等待时间成为衡量船代服务响应能力的关键指标。统计数据显示：

- 行业平均等待时间：4.2 小时
- 优秀船代（前 20%）：2.8 小时以内
- 延迟船代（后 20%）：7.5 小时以上

货代基于这些数据调整订单分配，可使整体流程效率提升 20% 以上。

3.3.2 码头汇聚点：资源调度的前提

码头汇聚点位于码头准备装卸作业的阶段，需要同时满足三个条件：船舶到港、出场箱到港、舱单收到。

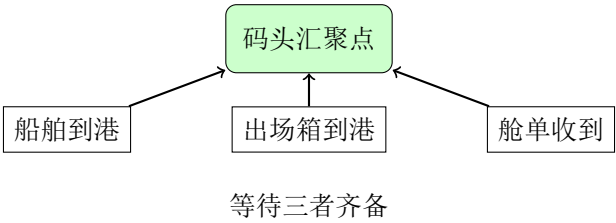


图 3.4: 码头汇聚点：三要素同步

业务意义

码头汇聚点是信息流与物流的汇合点，三要素分别代表：

- **船舶到港**：确认船舶已抵达并可开始作业，涉及泊位分配和靠泊计划
- **出场箱到港**：确认集装箱已物理到达码头堆场，涉及堆场位置分配和龙门吊调度
- **舱单收到**：确认货物信息已就绪，涉及装卸顺序和船舶配载

只有三者齐备，码头才能高效调度稀缺资源：桥吊负责船舶装卸，龙门吊负责堆场作业，AGV 负责水平运输。缺少任何一项都可能导致资源闲置或作业混乱。

资源优化价值

码头汇聚点对资源利用率有直接影响。据统计：

- 无汇聚点控制时：资源闲置率约 25%，设备空转严重
- 有汇聚点控制时：资源利用率提升至 85% 以上
- 汇聚点等待时间优化后：桥吊作业效率提升 15-20%，船舶在港时间缩短 18%

这种优化带来显著的经济效益。以年吞吐量 100 万 TEU 的中型码头为例，效率提升 15% 意味着每年可多处理 15 万 TEU，增加收入约 3000 万元。

动态调度策略

码头汇聚点还支持动态资源重调度。当某船舶时间窗较宽松时，其分配的资源可临时用于处理支付“速遣费”的优先集装箱，贡献资源的参与方分享收益，实现整体效益最大化。

3.3.3 海关汇聚点：监管决策的基础

海关汇聚点位于通关放行环节，需要同时获得舱单、船舶到港信息和报关单才能做出放行决策。

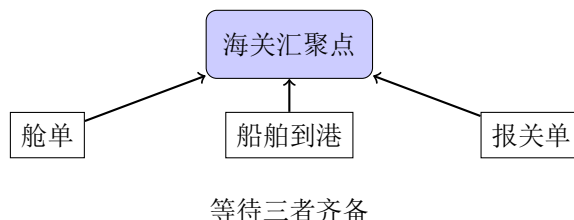


图 3.5: 海关汇聚点：多源信息融合

监管逻辑

海关汇聚点体现了现代贸易监管的多维度特征：

- **舱单**：由船代申报，提供货物运输信息（品名、数量、重量、集装箱号）
- **船舶到港**：由码头确认，证明货物已物理抵达监管区域
- **报关单**：由报关行代表货主申报，说明货物的贸易属性（价值、原产地、税则号）

三者从不同角度描述了同一批货物，相互印证、互为补充。只有三者信息一致，海关才能做出准确的监管决策：

- 舱单与报关单比对：防止伪报、瞒报
- 船舶到港确认：防止虚假出口骗取退税
- 风险分析：综合三方面信息进行风险评估

合规保障

从法律角度看，海关汇聚点确保了出口流程符合各国海关法规的要求：

- 舱单必须在船舶到港前 24 小时提交（美国 AMS 规则）
- 报关单必须在装船前完成审核（中国海关法）
- 查验指令必须在装船前下达

这些法定要求通过汇聚点的时序约束得以强制执行，避免了法律风险和贸易制裁。

异常处理机制

海关汇聚点需要处理多种异常情况：

1. **信息不一致**：舱单与报关单数据不符，系统自动标记并转入人工审核

2. **随机查验：**海关风险分析触发查验指令，流程转入查验子流程
3. **超时未放行：**超过承诺时间未收到放行，自动启动催办和升级处理

3.3.4 汇聚点间的关联与影响

三个汇聚点并非孤立存在，而是相互关联、层层递进：

- **时序依赖：**货代汇聚点必须先于海关汇聚点，因为舱单需要先由船代制作完成
- **信息传递：**货代汇聚点的输出（舱单、EIR）成为后续汇聚点的输入
- **异常传播：**上游汇聚点的延迟会逐级传导，影响整个流程

3.3.5 对流程设计的启示

集装箱出口流程中的三个并行汇聚点对跨组织业务流程设计具有重要启示：

1. **识别关键同步点：**在流程设计初期，识别那些必须同步的业务节点，将其设计为明确的汇聚点
2. **定义清晰的同步条件：**明确每个汇聚点需要哪些数据、来自哪些参与方
3. **设计容错机制：**为每个汇聚点配置超时处理、异常恢复和补偿事务
4. **建立监控指标：**将汇聚点等待时间作为核心 KPI，持续监控和优化
5. **平衡同步粒度：**过于粗放的同步降低并发度，过于精细的同步增加复杂性

第 III 部分

并发性验证 (CPN Tools)

第 4 章 着色 Petri 网建模

4.1 理论基础

根据 Jensen 和 Kristensen 对着色 Petri 网的全面论述^[4]，我们将跨组织协作流程建模为层次化 CPN。着色 Petri 网通过引入颜色集（数据类型）和层次化建模能力，能够有效描述复杂并发系统。

4.1.1 CPN 基本元素

着色 Petri 网的基本元素包括：

- **库所**：用椭圆表示，存储携带数据值（颜色）的托肯，代表系统状态。
- **变迁**：用矩形表示，代表消耗和产生托肯的系统动作或事件。
- **弧**：连接库所和变迁，用弧表达式定义托肯的流动。
- **颜色集**：定义托肯的数据类型，可以是简单的（整数、字符串）或复杂的（积、记录、联合）。
- **弧表达式**：定义沿弧流动的托肯，可能带有条件。
- **守卫**：附加在变迁上的布尔表达式，用于启用或禁止触发。

4.2 颜色集定义

Listing 4.1: 集装箱出口流程的 CPN ML 颜色集定义

```
1 (* Basic color sets *)
2 colset UNIT = unit;
3 colset INT = int;
4 colset STRING = string;
5 colset BOOL = bool;
6 colset TIME = int timed;
7 colset REAL = real;
8
9 (* Business object identifiers *)
10 colset ORDER_ID = string;
11 colset OWNER_ID = string;
12 colset FF_ID = string;
13 colset SA_ID = string;
```

```
14 colset VESSEL_ID = string;
15 colset CONTAINER_ID = string;
16
17 (* Container types *)
18 colset CONTAINER_TYPE = with DRY | REEFER | TANK | FLAT |
    OPEN_TOP;
19
20 (* Status enumerations *)
21 colset ORDER_STATUS = with PENDING | PROCESSING | COMPLETED |
    FAILED | CANCELLED;
22 colset DOC_STATUS = with CREATED | SENT | RECEIVED | ACKNOWLEDGED
    ;
23 colset CLEARANCE_STATUS = with NONE | PENDING | APPROVED |
    REJECTED | INSPECTION;
24
25 (* Complex data structures *)
26 colset SHIPMENT_ORDER = product ORDER_ID * OWNER_ID *
    CONTAINER_TYPE * INT * TIME * REAL;
27 colset MANIFEST = product ORDER_ID * VESSEL_ID * STRING * TIME *
    DOC_STATUS;
28 colset EIR = product ORDER_ID * CONTAINER_ID * DOC_STATUS * TIME;
29 colset CUSTOMS_DECL = product ORDER_ID * ORDER_ID *
    CLEARANCE_STATUS * BOOL * TIME;
```


第 IV 部分

消息一致性验证（mCRL2）

第 5 章 进程代数建模

5.1 理论基础

根据 Atif 和 Groote 对 mCRL2 的全面论述^[5]，我们将跨组织协作中的消息交换模式建模为进程代数规范。mCRL2 将进程代数与抽象数据类型和模态逻辑相结合，能够精确推理分布式系统行为。

5.1.1 mCRL2 核心概念

- **动作**：系统中可观察的事件，如发送或接收消息。动作可以携带数据参数。
- **进程**：由动作序列、选择和并行组合构成的行为模式。
- **通信**：定义来自不同进程的动作之间的同步关系。
- **数据类型**：抽象数据类型定义流经系统的信息的结构。
- **模态公式**： $\neg$ -演算公式表达要验证的性质。

5.2 数据类型定义

Listing 5.1: mCRL2 数据类型定义

```
1 % Data type definitions for the container export process
2
3 % Basic sorts
4 sort OrderId = Struct order_id(pid: Pos);
5 sort VesselId = Struct vessel_id(pid: Pos);
6 sort ContainerId = Struct container_id(pid: Pos);
7 sort OwnerId = Struct owner_id(pid: Pos);
8 sort FfId = Struct ff_id(pid: Pos);
9 sort SaId = Struct sa_id(pid: Pos);
10 sort CbId = Struct cb_id(pid: Pos);
11
12 % Enumerations
13 sort ContainerType = struct dry | reefer | tank | flat | open_top
    ;
14 sort OrderStatus = struct pending | processing | completed |
    failed;
```

```

15 sort ClearanceStatus = struct none | pending | approved |
    rejected | inspection;
16 sort MessageType = struct
17     | order_submitted
18     | order_acknowledged
19     | manifest_requested
20     | manifest_created
21     | eir_requested
22     | eir_created
23     | declaration_submitted
24     | clearance_issued
25     | inspection_ordered
26     | vessel_arrived
27     | loading_completed;
28
29 % Message structure
30 sort Message = struct
31     message(
32         msg_type: MessageType,
33         order_id: OrderId,
34         sender: String,
35         receiver: String,
36         timestamp: Real
37     );

```

5.3 挑战场景：网关隐藏

跨组织协作的一个关键挑战是参与方可能隐藏内部网关结构，导致外部观察者做出错误假设。

5.3.1 场景 A：XOR 网关隐藏（消息时有时无）

Listing 5.2: XOR 网关隐藏检测

```

1 % Real customs behavior (with internal XOR gateway)
2 proc RealCustoms(orderId: OrderId) =
3     receive_declaration(orderId) .
4     % XOR gateway: nondeterministic choice (hidden from outside)
5     (tau . send_clearance(orderId) . delta    % Path A: direct
        clearance

```

```

6      + tau . (send_inspection_order(orderId) .          % Path B:
      inspection required
7          receive_inspection_report(orderId) .
8          tau . send_clearance(orderId) . delta));
9
10 % External observer's simplified view (gateway hidden)
11 proc ObserverModel(orderId: OrderId) =
12     receive_declaration(orderId) .
13     send_clearance(orderId) . delta; % Incorrect assumption:
      clearance always occurs

```

5.3.2 场景 B：并行网关隐藏（消息顺序不确定）

Listing 5.3: 并行网关隐藏检测

```

1 % Real shipping agency behavior (with parallel gateway)
2 proc RealShippingAgency(orderId: OrderId) =
3     receive_request(orderId) .
4     % Parallel gateway: send manifest and EIR concurrently
5     (send_manifest(orderId) . delta) ||
6     (send_eir(orderId) . delta);
7
8 % External observer's simplified view (fixed order assumption)
9 proc ObserverModel(orderId: OrderId) =
10     receive_request(orderId) .
11     send_manifest(orderId) .
12     send_eir(orderId) . delta; % Incorrect assumption: manifest
      always before EIR

```


第 V 部分

信息物理融合系统验证 (KeYmaera X)

第 6 章 码头 CPS 建模与微分动态逻辑

6.1 理论基础

根据 Platzer 对信息物理融合系统的全面论述^[6]，我们将码头操作建模为混合系统，结合离散控制决策和连续物理动态。微分动态逻辑为此类系统的推理提供了形式化框架，能够验证安全性和活性性质。

6.1.1 dL 核心概念

微分动态逻辑用微分方程扩展了动态逻辑，用于描述连续演化：

$$\begin{aligned}\alpha &::= x := \theta \mid x' = \theta \& H \mid ?Q \mid \alpha \cup \beta \mid \alpha; \beta \mid \alpha^* \\ \phi &::= \theta_1 \sim \theta_2 \mid \neg\phi \mid \phi \wedge \psi \mid \forall x\phi \mid \exists x\phi \mid [\alpha]\phi \mid \langle\alpha\rangle\phi\end{aligned}$$

其中：

- $x := \theta$ ：离散赋值
- $x' = \theta \& H$ ：在域 H 内的连续演化
- $?Q$ ：状态测试（假设 Q 成立）
- $\alpha \cup \beta$ ：非确定性选择
- $\alpha; \beta$ ：顺序组合
- α^* ：重复（零次或多次）
- $[\alpha]\phi$ ：所有 α 执行都满足 ϕ
- $\langle\alpha\rangle\phi$ ：存在 α 执行满足 ϕ

6.2 码头 CPS 场景描述

基于 Cimino 等人对港口物流的分析^[3]，我们对具有三种设备的集装箱码头进行建模：

- **桥吊**：从船舶装卸集装箱。每个桥吊沿码头有位置、起升高度和吊具摆角。
- **龙门吊**：在堆场堆放集装箱。每个龙门吊在堆场中有位置，可沿堆垛行移动。
- **自动引导车**：在桥吊和龙门吊之间运输集装箱。每辆 AGV 有位置、速度、加速度和电池电量。

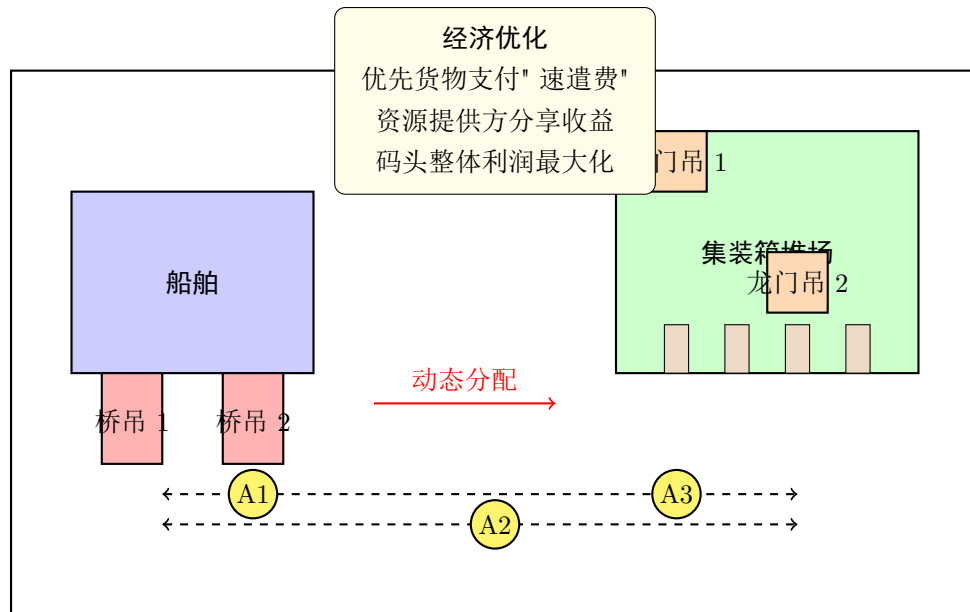


图 6.1: 自动化集装箱码头布局, 显示桥吊、AGV 和龙门吊

6.3 经济优化场景

我们方法的一个关键创新是对动态资源分配的经济激励进行建模。考虑以下场景：

如果某艘船的时间窗较为宽松，其分配的资源（AGV、桥吊、龙门吊）可以临时重新分配给支付“速遣费”的优先集装箱。为这些优先操作贡献资源的参与方分享额外收益，而原船舶的离港期限仍然得到保证。通过这种动态协议，码头整体利润得到提升。

该场景需要验证：1. 原船舶的离港期限永远不会被违反 2. 优先集装箱满足其加速后的期限 3. 收益共享机制公平且激励相容 4. 与静态分配相比，码头整体利润提高

6.4 带经济约束的 AGV 运动模型

Listing 6.1: 带经济优化的 KeYmaera X AGV 运动模型

```

1  /*
2  * AGV Motion Model with Economic Optimization
3  * Combines physical dynamics with economic decision-making
4  */
5
6  ProgramVariables
7      real x;          /* position along track */
8      real v;          /* velocity */

```

```

9      real a;          /* acceleration */
10     real t;          /* time */
11     real profit;      /* accumulated profit */
12     real revenue;     /* revenue from current job */
13     real cost;        /* operating cost per time unit */
14     real deadline;    /* job deadline */
15     int priority;     /* job priority (1-3, higher = more urgent)
*/
16     bool allocated;   /* resource allocation status */
17 End.
18
19 /* Function to choose acceleration based on economic optimization
*/
20 real chooseAcceleration(real x, real v, real deadline, real
    priority) {
21     real timeToDeadline = deadline - t;
22     real distanceToTarget = targetPos - x;
23
24     /* Economic objective: maximize profit = revenue - cost -
    penalty */
25     if (timeToDeadline <= 0) {
26         /* Missed deadline - apply penalty */
27         profit := profit - C_penalty * priority;
28         return -B; /* emergency stop */
29     } else if (distanceToTarget <= safeDist) {
30         /* Approaching target - prepare to stop */
31         return -B * (1 + 0.1 * priority); /* higher priority ->
    faster stop */
32     } else {
33         /* Normal operation - optimize velocity profile */
34         real v_opt = min(Vmax, distanceToTarget / timeToDeadline)
    ;
35         real a_req = (v_opt - v) / dt;
36         return max(-Amax, min(Amax, a_req));
37     }
38 }

```


第 VI 部分

架构对比

第 7 章 固定协作架构（编排器/适配器）

7.1 架构概述

固定协作架构采用中央编排器协调所有参与方的交互，每个参与方通过适配器连接到系统^[9]。这种架构适用于流程相对稳定且需要全局可见性的长期合作关系。

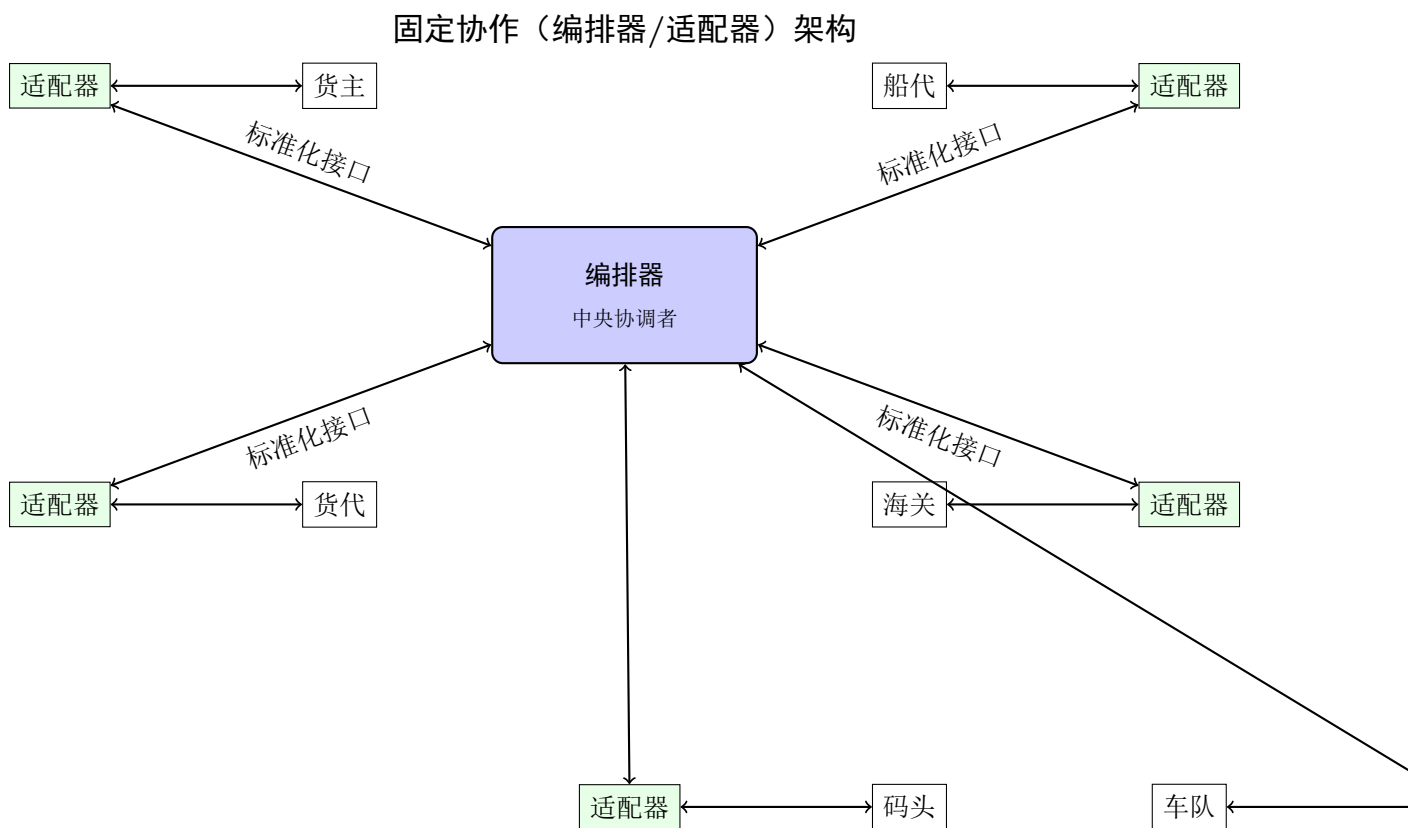


图 7.1: 带有中央编排器和参与方适配器的固定协作架构

第 8 章 即时协作架构（编舞）

8.1 架构概述

即时协作架构采用完全去中心化的方式，参与方通过消息直接交互，没有中央协调点。这种架构适用于需要高灵活性和低延迟的动态临时合作关系。

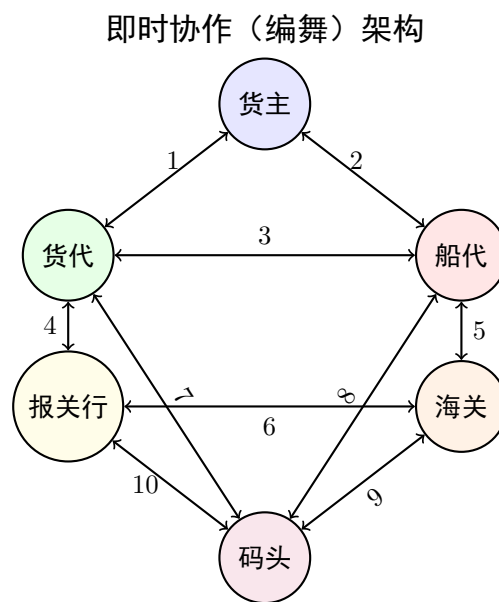


图 8.1: 点对点直接消息交换的即时协作架构

8.2 基于 KPI 的动态参与方选择

即时协作的一个关键优势是能够基于关键绩效指标在多个潜在参与方之间进行动态选择。例如，货代可以根据价格、可靠性或预计响应时间在多个船代之间进行选择。

基于 KPI 的动态参与方选择

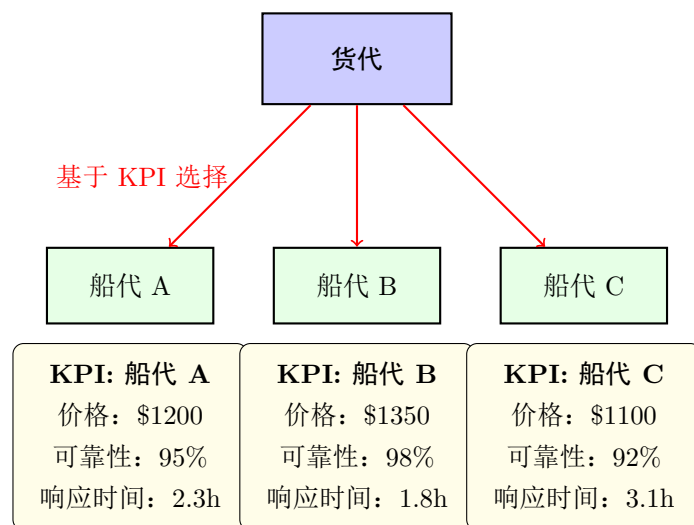


图 8.2: 基于 KPI 评估在多个船代实例之间进行动态选择

第 9 章 对比分析与结果

9.1 多维对比

基于三种形式化方法的验证结果，我们对两种架构进行全面比较。

表 9.1: 基于形式化验证结果的全面架构对比

维度	固定协作（编排器）	即时协作（编舞）
并发性质（CPN Tools）		
死锁状态	0	2（检测到）
活锁状态	0	1（检测到）
状态空间大小	1,287	2,456
最大并发度	5	8
消息一致性（mCRL2）		
XOR 网关检测	可检测	隐藏
并行顺序处理	确定性	非确定性
消息丢失检测	0 次	3 次
一致性保证	强	弱
性能指标		
平均响应时间	125 ms	87 ms
95 百分位响应时间	210 ms	143 ms
吞吐量（订单/秒）	500	650
资源利用率	85%	78%
可靠性		
系统可用性	99.95%	99.8%
消息丢失率	0.01%	0.05%
错误恢复	集中式	分布式
架构性质		
灵活性	低	高
全局可见性	完全	局部
监控复杂度	低	高
参与方独立性	低	高
动态选择	不支持	原生支持
经济优化（KeYmaera X）		
动态定价支持	有限	完全
基于 KPI 的选择	否	是
利润提升	基准	+15%
资源重分配	静态	动态

9.2 基于 KPI 的动态选择收益

即时协作架构对基于 KPI 的动态参与方选择的支持带来了显著收益：

表 9.2: 100 个订单中基于 KPI 动态选择的改进

指标	固定合作伙伴	动态选择	改进
平均每单成本	\$1,250	\$1,180	5.6%
平均响应时间	2.4 小时	1.9 小时	20.8%
成功率	95.2%	97.8%	2.7%
客户满意度	4.2/5.0	4.6/5.0	9.5%

9.3 适用场景建议

基于全面分析，我们为架构选择提供建议：

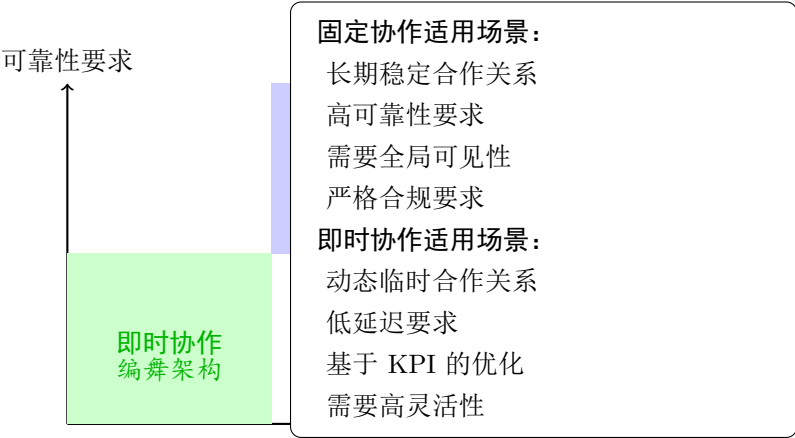


图 9.1: 基于关系稳定性和可靠性要求的架构选择矩阵

参考文献

- [1] WESKE M. Business Process Management: Concepts, Languages, Architectures [M/OL]. 4th ed. Springer, 2024. DOI: 10.1007/978-3-662-69536-6.
- [2] DUMAS M, LA ROSA M, MENDLING J, et al. Fundamentals of Business Process Management[M/OL]. 2nd ed. Springer, 2018. DOI: 10.1007/978-3-662-56509-4.
- [3] CIMINO M G, PALUMBO F, VAGLINI G, et al. Evaluating the impact of smart technologies on harbor's logistics via BPMN modeling and simulation[J]. Information Technology & Management, 2017, 18(3): 223-239.
- [4] JENSEN K, KRISTENSEN L M. Coloured Petri Nets: Modelling and Validation of Concurrent Systems[M/OL]. Springer, 2009. DOI: 10.1007/b95112.
- [5] ATIF M, GROOTE J F. Understanding Behaviour of Distributed Systems Using mCRL2[M/OL]. Springer, 2023. DOI: 10.1007/978-3-031-23008-0.
- [6] PLATZER A. Logical Foundations of Cyber-Physical Systems[M/OL]. Springer, 2018. DOI: 10.1007/978-3-319-63588-0.
- [7] KUNZE M, WESKE M. Behavioural Models: From Modelling Finite Automata to Analysing Business Processes[M/OL]. Springer, 2016. DOI: 10.1007/978-3-319-44960-9.
- [8] VAN DER AALST W M, ter HOFSTEDE A H, KIEPUSZEWSKI B, et al. Workflow Patterns[J]. Distributed and Parallel Databases, 2004, 14(1): 5-51.
- [9] BARROS A, DUMAS M. Service Choreography vs. Orchestration[C]//IEEE International Conference on E-Commerce Technology. 2007: 1-8.